

UNIVERSITY OF CALIFORNIA SAN DIEGO

Hidden Meaning: Interpretations of AutoRegressive Transformer Hidden States

A Thesis submitted in partial satisfaction of the requirements
for the degree Master of Science

in

Computer Science

by

Akshay Murali

Committee in charge:

Professor Yoav Freund, Chair
Professor Taylor Berg-Kirkpatrick
Professor Julian McAuley

2026

©

Akshay Murali, 2026

All rights reserved.

The Thesis of Akshay Murali is approved, and it is acceptable in quality and form for publication on microfilm and electronically.

University of California San Diego

2026

DEDICATION

I dedicate my work to my friends and family, whose unwavering support, guidance, and encouragement sustained me throughout this journey. Their patience, belief in me, and constant reassurance provided the comfort and motivation I needed to bring this work to completion.

I am also deeply grateful to my thesis advisor, Professor Yoav Freund, whose wisdom, mentorship, and thoughtful guidance shaped both this project and my growth as a researcher. His dog, Mishmish, with her joyful presence, also offered great comfort and relief after many long hours of work. Without their insight, support, and encouragement, this thesis would not have been possible.

EPIGRAPH

“Words differently arranged have a different meaning, and meanings differently arranged have different effects.”

Blaise Pascal

TABLE OF CONTENTS

THESIS APPROVAL PAGE	iii
DEDICATION	iv
EPIGRAPH	v
TABLE OF CONTENTS	vi
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
VITA	xi
ABSTRACT OF THE THESIS	xii
INTRODUCTION	1
Chapter 1 SETUP & BACKGROUND	6
Chapter 2 DATASET & DATA GENERATING PROCESS	10
Chapter 3 EXPLORATORY DATA ANALYSIS (EDA).....	14
Chapter 4 METHODOLOGY	18
Chapter 5 IMPLEMENTATION	21
Chapter 6 EXPERIMENTAL RESULTS	23
I. Overall Accuracy Across Layers and Context Levels	23
II. Per-prompt Test Accuracy	24
III. Layer-to-Layer and Context Level-to-Context Level Patterns	26
IV. First-token Generations with Single Token Context	27
V. Hyperparameter Optimization via Bayesian Search	28
VI. XGBoost Feature Importance (Hidden Dimension) Analysis	29
Chapter 7 DISCUSSION OF RESULTS	35
Chapter 8 RELATED WORK	45

Chapter 9 FUTURE WORK	51
REFERENCES	56

LIST OF FIGURES

Figure 1.1: AR transformer architecture overview. Copyright obtained and borrowed under fair use from https://www.researchgate.net/figure/Decoder-only-Transformer-architecture-The-input-to-the-decoder-is-tokenized-text-and_fig3_373183262	7
Figure 1.2: AR transformer generative procedure. Borrowed under fair use from https://learn.deeplearning.ai/courses/how-transformer-llms-work/lesson/hrpcy/understanding-language-models:-transformers	7
Figure 3.1: Distribution of token lengths of generated stories.....	15
Figure 6.1: Train Accuracy Heatmap.....	23
Figure 6.2: Test Accuracy Heatmap	24
Figure 6.3: a) – j) Test accuracy heatmaps for prompts 1-10, respectively.....	24
Figure 6.4: a) Average difference in test accuracy between consecutive layers. b) Average difference in test accuracy between current layer and layer 0.	26
Figure 6.5: a) Average difference in test accuracy between consecutive generation levels. b) Average difference in test accuracy between current generation level and the first generation level.....	27
Figure 6.6: a) Heatmap of overall train and test accuracy across all layers when using XGBoost to perform prompt prediction on first token generation hidden states with single token context. 27	
Figure 6.7: Heatmap of train accuracy after hyperparameter optimization is conducted.	28
Figure 6.8: Heatmap of test accuracy after hyperparameter optimization is conducted.....	29
Figure 6.9: Heatmap of feature importances as determined by XGBoost when making predictions on the test set.....	30
Figure 6.10: Comparison between important features determined using Shapley values. Shown here are the most important features determined when using hidden states collected at layer 1 during the generation of the a) first token vs. those collected during the generation of the b) 2 nd and c) 25 th tokens.	31
Figure 6.11: a) – j) Heatmap of most important features used for prediction on the test set across all layers and generation levels.	32
Figure 6.12: Heatmap showing the mean of the five most important features across generation levels 2-400 and all layers.....	34
Figure 7.1: a) Heatmap of overall train and test accuracy across all layers for secondary experiment concerning first token generation with minimal context. b) Heatmap of prompt 1 train and test accuracy across all layers for secondary experiment concerning first token generation with minimal context. c) Heatmap of prompt 2 train and test accuracy across	41

LIST OF TABLES

Table 2.1: Input prompts used for generating stories.	13
--	----

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AR	AutoRegressive
CNN	Convolutional Neural Network
FFN	Feedforward Network
LM	Language Model
LLM	Large Language Model
MLP	Multilayer Perceptron
SHAP	SHapley Additive exPlanations
XAI	Explainable Artificial Intelligence

VITA

2023 Bachelor of Science in Data Science, University of California San Diego

2026 Master of Science in Computer Science, University of California San Diego

FIELD OF STUDY

Major Field: Computer Science
Studies in Interpretability
Professor Yoav Freund

ABSTRACT OF THE THESIS

Hidden Meaning: Interpretations of AutoRegressive Transformer Hidden States

by

Akshay Murali

Master of Science in Computer Science

University of California San Diego, 2026

Professor Yoav Freund, Chair

Autoregressive transformers are the dominant architecture in modern language models (LMs), yet the relationship between their inputs and outputs remains poorly understood. This limited interpretability contributes to user distrust, hallucinations, and concerns about their “black-box” nature. To address this, we investigate how input information is represented and transformed across layers during text generation.

We analyze a simple autoregressive transformer by measuring how strongly it retains prompt information as it generates tokens. Hidden states from each layer and generation step are collected and used as features in a supervised prompt-recovery task. Specifically, we train an XGBoost classifier to predict the original prompt from these hidden states and examine feature importance using SHAP values.

Our results show that prompt information in hidden states diminishes as more tokens are generated. However, across layers, prompt-related information cumulatively increases. This pattern holds regardless of the prompt or output length. We also observe an initial drop in predictive accuracy in early layers, followed by recovery in later layers, suggesting that early representations are refined and compounded rather than discarded. Feature importance analysis reveals that specific hidden state dimensions consistently encode prompt-relevant information across layers and generation steps.

These findings help identify which layers most directly shape output based on input. Such insights may guide model pruning and compression by removing less impactful components, enabling smaller, more energy-efficient models. Improved efficiency could reduce inference costs and the environmental impact of deploying large-scale language models.

INTRODUCTION

Interpretability of language models (LMs), known colloquially as Explainable Artificial Intelligence (XAI), concerns our ability to understand and trace how a model produces a particular output from a given input (Doshi-Velez & Kim, 2017; Linardatos et al., 2021). In classical machine learning, interpretability is comparatively straightforward. For simple linear models or tree-based architectures, the conclusion is derived directly from a simple underlying function or a series of logical steps (Hsieh et al., 2024; Linardatos et al., 2021). However, in the case of LMs, billions of parameters across many layers influence the decision-making process of the model. As a result, these models have become unintuitive and too complex beyond human understanding, and improving their interpretability has become a central research challenge (Fantozzi & Naldi, 2024; Mumuni & Mumuni, 2025; Palikhe et al., 2025; Zhao et al., 2023).

One promising direction towards greater interpretability lies in reducing architectural complexity. If certain components of a deep model contribute minimally in transforming the input to generate output, consolidating or removing them entirely could yield a system that is not only smaller and more efficient but also easier to understand (Blalock et al., 2020; Frankle & Carbin, 2019). Evidence from other domains, such as previous work done on vision models, supports this idea. Kim et al. (2024) and Han et al. (2016) have shown that layer pruning and merging is plausible and effective in convolutional neural networks (CNNs) for both reducing the size of the model while retaining performance. These findings suggest that deep architectures may contain structural redundancies, in which there exist layers whose contributions overlap or are otherwise nonessential. Translating this insight to autoregressive transformers raises a

compelling question: are all transformer layers equally necessary for mapping input prompts to generated outputs, or do some layers play a disproportionately important role?

Several techniques already aim to reduce the effective size of large language models. Low-Rank Adaptation (LoRA) introduces trainable low-rank matrices into pre-trained transformers, enabling efficient fine-tuning with far fewer parameters than full model updates (Hu et al., 2021). Although LoRA does not explicitly remove layers, it demonstrates that only a small subspace of the parameter space may be required to adapt a model to new tasks. Model distillation is another method that significantly reduces model size. In this framework, a larger model is employed to teach a smaller model how to respond to queries. Through this knowledge transfer, the smaller model learns to approximate the teacher’s outputs by looking at sample inputs and corresponding outputs produced by the larger model (Hinton et al., 2015). The public release of compact language models, such as those introduced by DeepSeek, illustrates how distillation can produce smaller systems that retain strong generative performance. Together, these approaches highlight the possibility that larger models contain compressible or redundant structure (Sun et al., 2026, Zhang et al., 2024).

However, despite these advances, a fundamental question remains insufficiently explored: which specific layers in an autoregressive transformer are most critical for preserving and transforming prompt information during generation? Compression techniques demonstrate that models can be made smaller, but they often do so without a detailed understanding of how information flows through the network. To move beyond empirical compression and towards

principled interpretability, we must identify where and how the relationship between input and output is encoded internally.

To address this gap, we conduct a focused study on a small language model trained on a collection of children’s stories. This model provides a controlled and semantically coherent setting in which narrative structure and thematic continuity are central to output generation. We construct a dataset by prompting the model with ten distinct story openings, each designed to mimic the introduction to a unique children’s tale. For each prompt, the model generates a full story continuation. During this autoregressive generation process, we store the hidden states from every transformer layer at each generation step.

These hidden states represent the intermediate embeddings the model uses to compute the next token probabilities. Though they are high-dimensional and not directly interpretable, they contain the internal representations through which the model processes context (Lipton, 2016; Vaswani et al., 2017). Using these hidden states collected during the generation of the stories, we ask, “which layers, and at which points during generation, retain the most information about the original prompt?” By answering this question, we aim to localize prompt-related information within the model’s depth and across its temporal generation trajectory.

More specifically, we want to investigate whether prompt information is preserved, amplified, or diluted as the story grows longer. Autoregressive generation proceeds token by token, conditioning each new prediction on the previously generated sequence (Radford et al., 2019). Intuitively, one might expect that as the narrative expands, the influence of the initial

prompt could weaken, especially if later tokens introduce new characters or plot elements. Alternatively, it is possible that deeper layers continuously integrate and reinforce prompt-relevant features, ensuring thematic coherence throughout the story.

To quantify these effects, we analyze hidden states as features in a downstream predictive task. By attempting to recover the original prompt from hidden state representations at different layers and generation steps, we obtain a measure of how much prompt information is encoded at each stage. If hidden states from a particular layer allow accurate prompt identification, this suggests that the layer strongly represents input-related information. Conversely, poor predictive performance implies that the layer’s representations are more focused on recently generated context or other transformations unrelated to the original prompt.

Through this framework, we examine several key questions. First, does increasing generation length systematically degrade the model’s internal representation of the original prompt? Second, are certain layers more robust to this potential degradation? Third, do early layers primarily process surface-level lexical features, while middle or deeper layers integrate higher-level narrative structure tied to the prompt? Finally, is the distribution of prompt information uniform across layers, or does it concentrate in specific regions of the network?

By localizing prompt-related information within the model’s architecture, we aim to clarify the functional roles of different layers. If we can identify layers that consistently encode strong input–output relationships, these layers may be central to the generative process. Conversely, layers that contribute little to maintaining prompt information may be candidates for

pruning or merging. Such insights could inform more targeted model compression strategies, reducing redundancy while preserving the components most essential for coherent generation.

Beyond architectural optimization, this line of inquiry contributes to broader interpretability goals. Understanding how and where a model encodes prompt information helps demystify the mechanisms underlying narrative continuity and thematic consistency. It also provides a concrete step toward bridging the gap between opaque distributed representations and human-interpretable internal structure.

While substantial progress has been made in compressing and fine-tuning language models, a detailed understanding of how prompt information propagates through their layers remains limited. By analyzing hidden states during story generation and evaluating their capacity to recover original prompts, we seek to identify the layers most responsible for linking input to output. This approach not only advances interpretability research but also lays the groundwork for principled model reduction, ultimately contributing to more transparent, efficient, and trustworthy language systems.

Chapter 1 SETUP & BACKGROUND

To evaluate our hypotheses, we utilize the llamatales-33M model developed by Ivan Lee, which shares the same architectural design as Meta’s Llama3 model but at a dramatically reduced scale, making it particularly suitable for interpretability analysis. By working with a compact model, we are able to collect and analyze hidden representations across all layers and generation steps without the computational barriers encountered with larger language models. This allows us to perform a more thorough analysis of the information stored and propagated through its minimal setup.

The model was trained using the LlamaTales dataset, an adaptation of the TinyStories dataset (an open-source collection of children’s stories with a limited vocabulary), but generated using the Llama3-8B-Instruct model (Lee & Berg-Kirkpatrick, 2025). Despite its small size, the model generates high-quality output on relevant prompts associated with children’s stories. Thus, to ensure that our data is well-represented by the model’s vocabulary, we formed our dataset via ten instructive prompts regarding the generation of children’s stories.

To dive deeper into the inner mechanisms of autoregressive transformers, we must first understand how these models generate tokens more generally at inference-time. Autoregressive transformers generate tokens sequentially, utilizing previously generated or user-provided tokens as context to influence the generation of the next token.

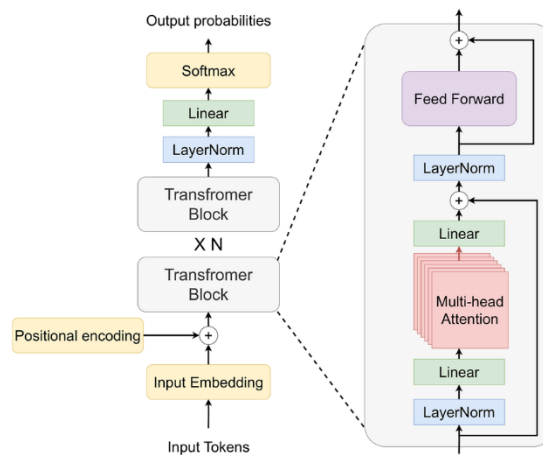


Figure 1.1: AR transformer architecture overview. Copyright obtained and borrowed under fair use from https://www.researchgate.net/figure/Decoder-only-Transformer-architecture-The-input-to-the-decoder-is-tokenized-text-and_fig3_373183262.

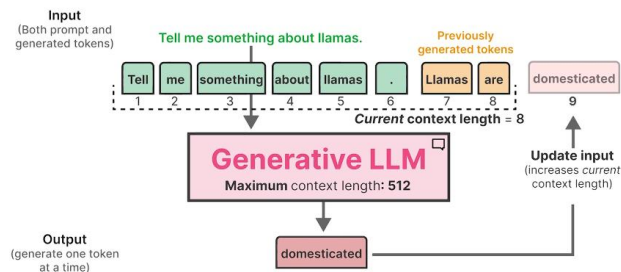


Figure 1.2: AR transformer generative procedure. Borrowed under fair use from <https://learn.deeplearning.ai/courses/how-transformer-llms-work/lesson/hrpcy/understanding-language-models:-transformers>.

Initially, the model embeds previously generated tokens and user-provided context into a numerical representation:

$$x_i = E(t_i) + p_i,$$

where t_i is the i th token, $E(\cdot)$ is the embedding lookup matrix, p_i is the positional encoding, and x_i is the input representation passed into the transformer.

Here, each token, t_i , is mapped to a continuous vector representation via an embedding matrix, and positional encodings, p_i , are added to preserve token order within the sequence.

Next, the embedded tokens are passed through a series of transformer blocks, consisting mainly of masked multi-head self-attention and a feedforward network (FFN), typically a multilayer perceptron (MLP). The output of one transformer block is sequentially fed into the next, gradually transforming and refining token embeddings into contextual representations. The output of the final transformer block undergoes a final linear projection followed by a softmax operation, which produces vectors that can be mapped to a probability distribution over the model's vocabulary.

To enforce sequential generation, autoregressive transformers utilize masked self-attention, a process that ensures only previous token information is considered to generate future tokens. Furthermore, in each stage (layer) of the token generation process, the model utilizes an evolving key-value (KV) cache that stores searchable, dimensionally-reduced projections of intermediate attention representations produced in previous generation steps as well as in previous layers for the current token generation. This avoids redundant computations and

reduces the size of the activations (hidden states) passed between each layer while also preserving important contextual information (Vaswani et al., 2017).

The projected values stored in the KV cache are used in the calculations of hidden states and influence the overall prediction of the model. However, the hidden states themselves are of particular interest and importance due to their role in directly transporting information between layers in the token generation process, containing increasingly-abstracted information ranging from direct encodings of words to semantic and syntactical information relating words to each other. These are the values that contain the crucial information of past generations that are immediately used to generate future tokens, so analyzing their role in relating input to output is important in determining which layers and what levels of generation most directly lead us from the initial input to our final output.

Our model consists of nine layers, with a primary token embedding layer followed by eight transformer blocks. The final output layer that applies a softmax over the linearly-projected hidden states to suggest, out of the tokens in the model's vocabulary, what should likely be the next token in the output is not included in this count (Lee & Berg-Kirkpatrick, 2025).

Chapter 2 DATASET & DATA GENERATING PROCESS

Our project focuses on forming a relationship between the output of a language model and its underlying input. By collecting the hidden states passed through each layer of the model to produce output tokens, we look to highlight which layers of the model most strongly complete this relationship and whether this relationship is degraded through long-term generation. Collecting hidden states during forward passes allows us to capture those layers’ states that hold enough information to relate back to the original prompt from each generated token output. In other words, notions of the original prompt are deeply ingrained into the layer’s hidden states. To assess the degree to which this is the case, ten prompts were used to produce 1000 stories (10,000 stories total), of variable length, with a max token limit of 512. We choose a max token limit of 512 because that is the prescribed context window of our language model (Lee & Berg-Kirkpatrick, 2025). The ten prompts used are shown in Table 2.1.

While generating a story, the hidden states were collected and saved to npz files, with separate files created for each prompt. Within each file, hidden state arrays corresponding to individual story generations are indexed sequentially using keys labeled “arr_0” through “arr_999”, corresponding to the 1000 stories for that prompt.

At the beginning of story generation, the model uses the entire prompt as context when predicting the first generated token. In this initial step, the KV cache is not yet populated with prompt information, and the hidden states reflect the full prompt sequence being processed. The structure of the hidden state data at this stage is described by the following dimensions:

$$l_{\text{story}} \times L \times B \times l_{\text{prompt}} \times d_{\text{model}}.$$

Here, the l_{story} (“story length”) dimension represents the number of tokens ultimately generated for that story. Because the model generates tokens sequentially until a stopping condition is reached and an end-of-sequence (EOS) token is generated, this value remains variable across examples. The second dimension, L (# of layers), corresponds to the nine transformer layers in the model. Each layer produces its own hidden representation of the current context, allowing us to observe how the internal representation evolves as information passes through the network. The batch size dimension (B) has a constant value of one because stories are generated individually, while the l_{prompt} (prompt length) dimension reflects the number of tokens present in the initial context and is variable based on the length of the prompt (either nine for prompts 1-9 or ten for prompt 10). Finally, the hidden dimension (d_{model}) corresponds to the size of the hidden vector produced by each transformer layer and has a constant value of 512 for our model.

Once the first token has been generated, the structure of the hidden state data changes slightly. During subsequent generation steps, the model no longer processes the full prompt sequence again. Instead, it relies on the previously generated token combined with the cached

representations of earlier tokens stored in the KV cache. As a result, the hidden state tensor for each subsequent token has the following structure:

$$l_{\text{story}} \times L \times B \times l_{\text{context}} \times d_{\text{model}}.$$

In this case, the l_{context} (context length) dimension is equal to one because the model processes only the most recently generated token while referencing prior tokens through the KV cache. Although the explicit context dimension becomes smaller, the hidden state vectors still contain information derived from the entire preceding sequence due to the cached attention representations. All other dimensions remain the same as before. Together, these hidden state tensors form the primary dataset used in our analysis.

In addition to saving the hidden state data, a metadata table was created to facilitate data loading and exploratory data analysis (EDA). This table was stored as a CSV file containing the prompt label, the full prompt text, the generated story, the file path to the associated hidden state data, and the final length of the generated story for each generated example.

Table 2.1: Input prompts used for generating stories.

Prompt Number (Label)	Input Prompt
1	“Once upon a time there was a dragon”
2	“Once upon a time there was a princess”
3	“Once upon a time there were two children”
4	“Once upon a time there was a prince”
5	“Once upon a time there was a frog”
6	“Once upon a time there was a king”
7	“Once upon a time there was a queen”
8	“Once upon a time there was a wolf”
9	“Once upon a time there was a genie”
10	“Once upon a time there was a poor boy”

Chapter 3 EXPLORATORY DATA ANALYSIS (EDA)

To better understand the structure of the generated dataset and ensure that it was suitable for long-range token analysis, we conducted a preliminary examination of the length distribution of the generated stories. This analysis focused on determining whether the number of tokens produced per story varied significantly across prompts or whether generation lengths were relatively consistent regardless of the initial prompt. Establishing this distribution was important because large imbalances in story length could bias later analyses that examine how hidden states evolve over longer generation sequences.

To perform this evaluation, we calculated the total number of tokens in each generated story and compared these values both within individual prompt groups and across the entire dataset. This allowed us to observe whether certain prompts tended to produce longer narratives or whether the model generated stories of similar lengths across all prompts. By examining both per-prompt statistics and the overall distribution, we could determine whether the dataset maintained a relatively balanced structure.

The results of this analysis indicate that the majority of generated stories fall within a relatively short range, with most narratives containing fewer than 400 tokens. This pattern appears consistently across all prompts, suggesting that no single prompt systematically produced significantly longer or shorter outputs. While some variation in length is present due to the autoregressive nature of generation and the model’s natural stopping behavior, the overall distribution remains tightly clustered around shorter sequences. This finding is beneficial in

ensuring that we have a balanced dataset as we proceed to perform length-based analyses of the generated stories.

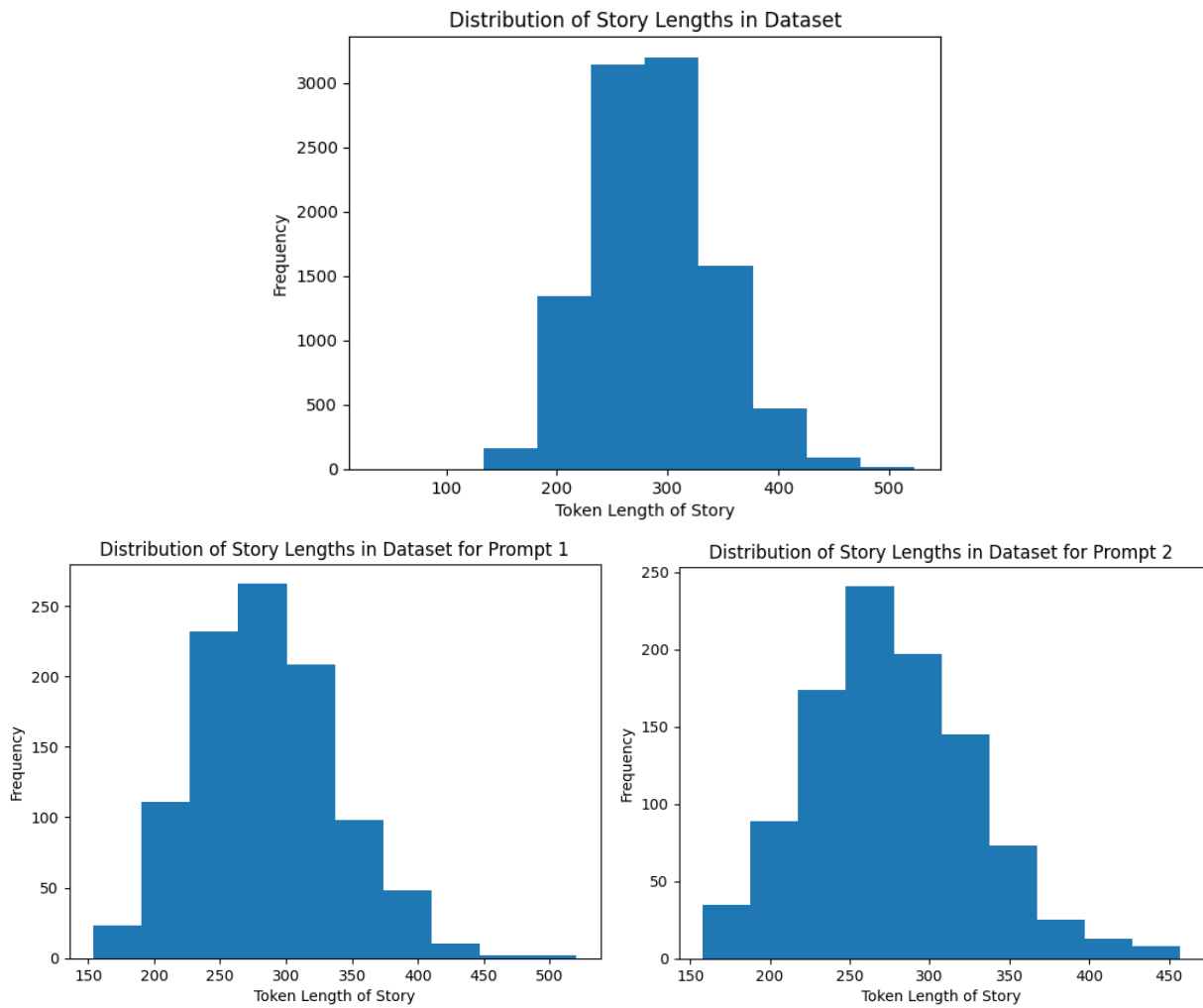


Figure 3.1: Distribution of token lengths of generated stories.

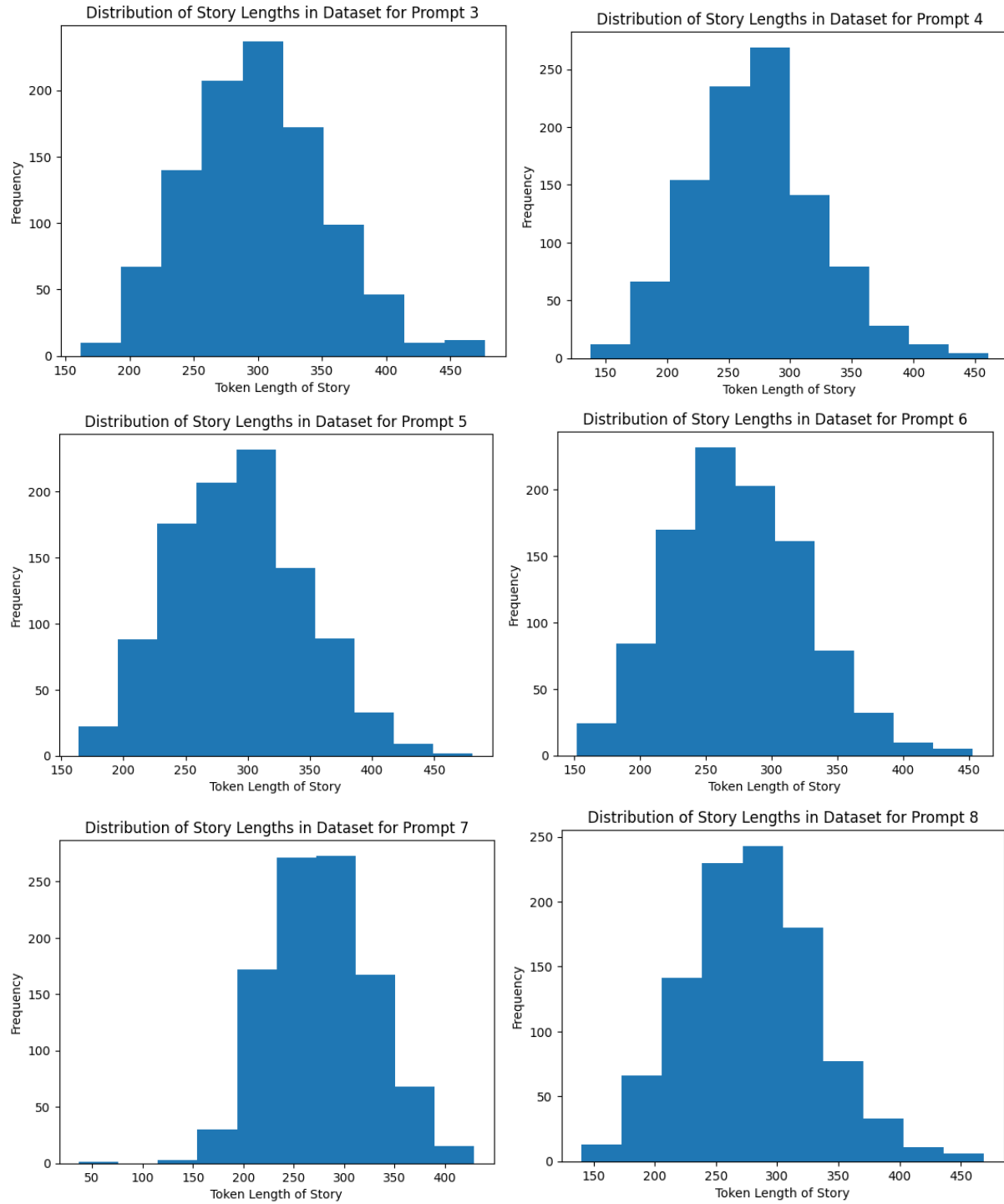


Figure 3.1, continued: Distribution of token lengths of generated stories.

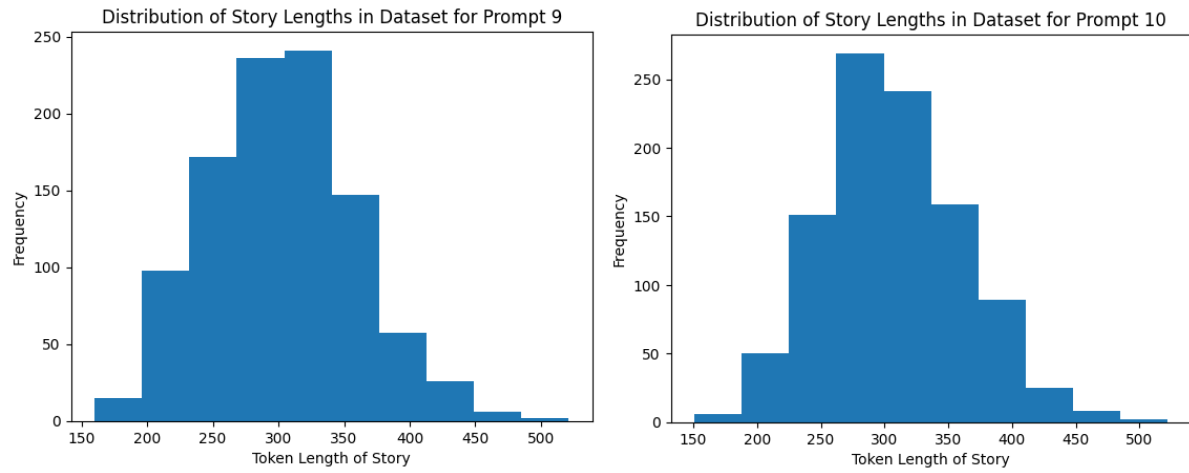


Figure 3.1, continued: Distribution of token lengths of generated stories.

Chapter 4 METHODOLOGY

By examining the hidden states produced at each layer during token generation, we aim to identify where within the model’s internal architecture the information connecting the prompt to the generated output token is most clearly represented. We seek to evaluate how well these representations encode information about the original prompt and uphold this relationship as the model generates longer sequences. Thus, the primary objective is to see which layers obtain the best accuracy when predicting the original prompt based solely off of the hidden states that it produces.

Using the collected hidden state tensors, we determine, more generally, whether later layers retain as much information about the prompt as earlier layers, and whether this information dwindles as more generated context is added to the model. In order to do this, we frame the problem as a supervised classification task. Specifically, we attempt to predict which prompt produced a given hidden state representation. If a hidden state contains strong information about the original prompt, a classifier should be able to correctly identify the prompt from that representation. Conversely, if the prompt information has largely faded from the representation, classification accuracy should decrease.

As our predictive model, we utilize a gradient boosted classifier via XGBoost. We choose this classifier mainly due to its strong predictive performance on tabular data with high dimensionality (Chen & Guestrin, 2016; Grinsztajn et al., 2022). Furthermore, the model is highly interpretable using external tools, which allow us to analyze directly which features (hidden dimensions) contribute most strongly to the classification decisions. In our final analysis,

we utilize SHapley Additive exPlanation (SHAP) values to estimate the importance of individual hidden state dimensions, offering further insight into how prompt information is encoded within the model (Lundberg et al., 2017, 2018, 2020).

We load our hidden states from npz files by layer and generation level. We index first by the generation depth we are currently analyzing followed by the specific layer. This results in vectors that are of shape $1 \times 1 \times 1 \times l_{\text{context}} \times 512$. For analysis regarding the generation of the first token, our core dataset consists of hidden state tensors of shape $(1 \times 1 \times 1 \times 9 \times 512)$ that are flattened and treated as 4608-dimensional vectors. Prompt 10 hidden states for the generation of the first token, which had dimension 10×512 , were indexed to only include vectors corresponding to the last 9 tokens in the prompt. Simple 512-dimensional vectors were used in later token generations, which had a context length of 1. Finally, our label set consists of prompt IDs with possible values ranging between 1 and 10, corresponding to the prompt used to generate the story.

In order to train the classifier, our dataset is first divided into training and test subsets. We perform an 80-20 split via indexed splitting, whereby the first eighty percent of examples corresponding to each prompt belong to the training set and the final twenty percent of examples belong to the test set.

With this framework in hand, we train an XGBoost classifier using histogram-based GPU acceleration and a default set of hyperparameters. Our objective is to minimize the classification error on our multi-class training set. After training the classifier, we evaluate its predictive

accuracy on both the training and test sets by comparing classifications to their ground truth values. We gather results on the overall dataset along with evaluation of predictive accuracy on individual prompts. This process is repeated across all layers and generation depths. Finally, using the collected results, we conduct a two-dimensional layerwise evaluation across generation levels and detail any visible patterns.

Chapter 5 IMPLEMENTATION

We ran a series of experiments using XGBoost in order to obtain conclusory results across all layers and levels of generation.

The primary set of experiments focused on gathering results using flattened hidden states across generation levels and layers. We indexed hidden states for each story in the generation of the 1st, 2nd, 25th, 50th, 75th, 100th, and so on in 25 token increments up to the generation of the 400th token. We only collected hidden states up to the 400th token because after the 400th token, a class imbalance was apparent, where some prompts did not have any examples with more than 400 tokens that could be gathered for the test set. We then gathered hidden states for each layer across these stratified generation levels. These hidden states were then passed through XGBoost with a default set of hyperparameters, as previously described, in order to obtain an accuracy score in predicting the original prompt from the hidden states.

The second set of experiments focused on analyzing the hidden states used to generate the first token. Because these hidden states encode the entire prompt with a dimension of either 9 or 10 x 512, depending on the length of the prompt, we seek to use only the hidden state corresponding to the encoding of the last token in the prompt in order to mimic how subsequent tokens in the story are generated. After collecting hidden states for each layer, we index the last token of context, obtaining hidden states of dimension 1x512 that now contain information about the last prompt token only.

Due to XGBoost’s tendency to overfit on the training set (Friedman, 2001), our third experiment involved applying the XGBoost algorithm to the hidden vectors in the training set via the Optuna framework, thereby conducting a Bayesian search of the model hyperparameters to both boost test performance as well as reduce the perfect accuracy achieved on the training set. For hyperparameter tuning, we also employ a validation set that serves to persuade the model from overfitting by prioritizing the accuracy of the validation set over the training set. Applying early stopping and reducing model complexity to prevent overfitting, we obtained a set of hyperparameters that reached close to optimal results for our use case. Supervised learning via XGBoost on the hidden states in the training set using this set of hyperparameters was performed and results on both our train and test set were collected.

The fourth and final experiment involved investigation into the Shapley values generated while conducting inference on the test set. By analyzing XGBoost’s feature importance, we want to explore whether certain hidden dimensions are more prone to or more important in storing prompt information than others. We also want to see whether there are patterns of certain hidden dimensions carrying the greatest amount of prompt information across both layers and generation levels. In this setting, we gather the most important feature and the mean of the top five most important features when our XGBoost classifier is applied to generate predictions across layers and generation levels on the test set. Similar to evaluation of predictive accuracy, we conduct a two-dimensional layerwise analysis of feature importance across generation levels and detail any patterns that may arise.

Chapter 6 EXPERIMENTAL RESULTS

The following section shows the results of the experiments described in the chapter entitled “Implementation”.

I. Overall Accuracy Across Layers and Context Levels

We first present the overall results of applying XGBoost with a default set of hyperparameters to the raw vectors in our train and test sets as a heatmap of layer vs. context (generation) level.

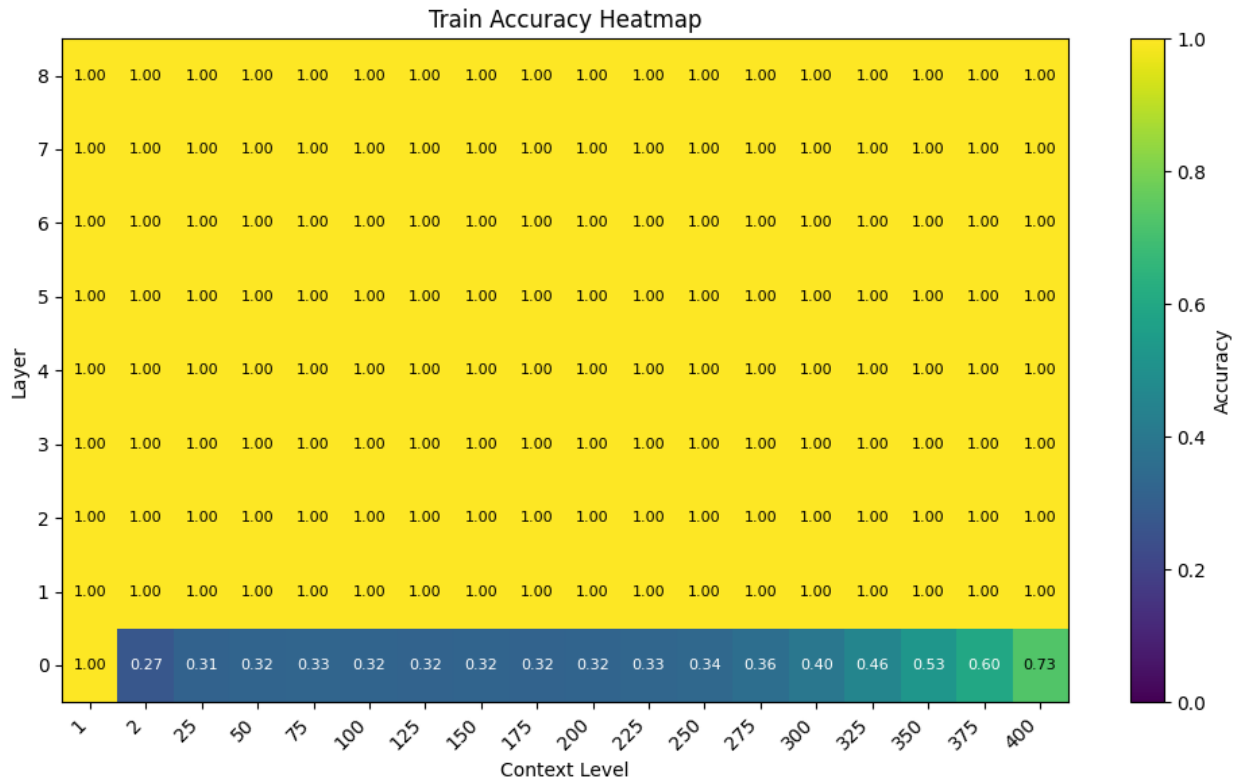


Figure 6.1: Train Accuracy Heatmap

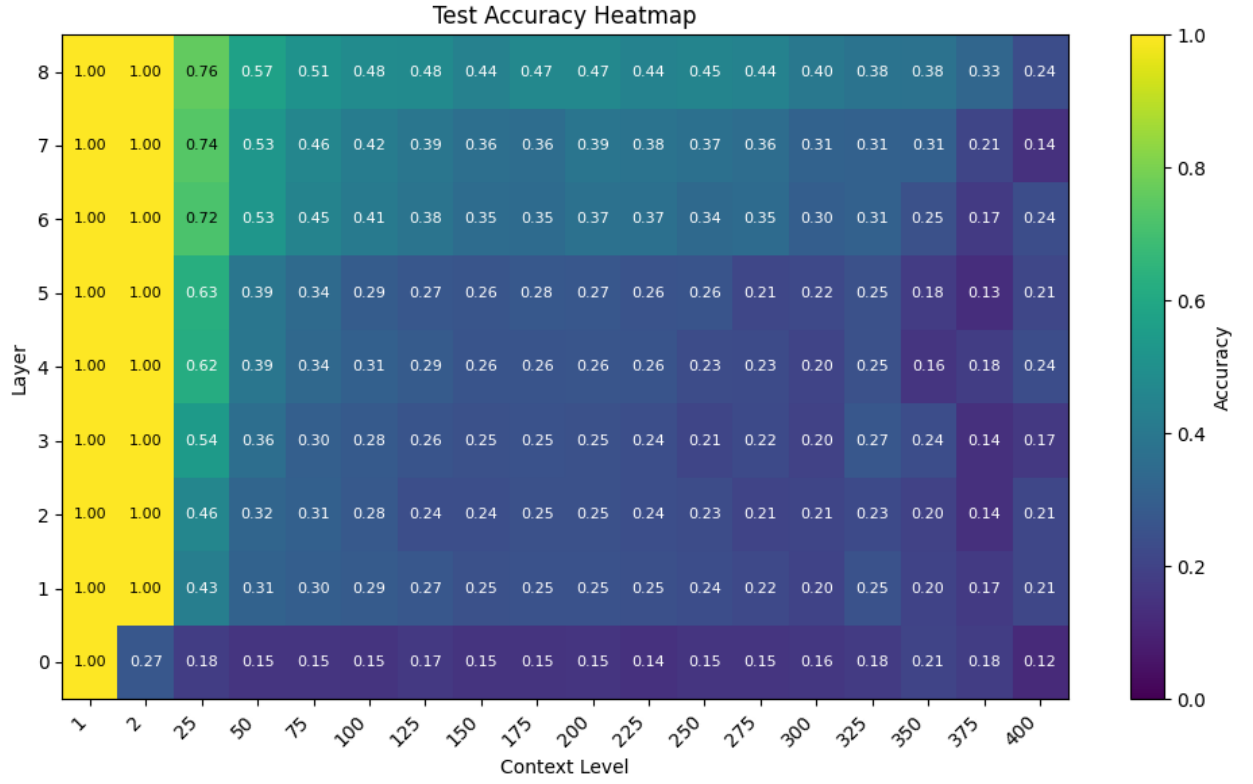
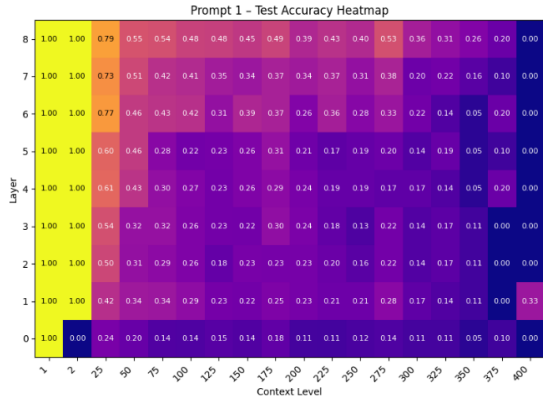


Figure 6.2: Test Accuracy Heatmap

II. Per-prompt Test Accuracy

Next, per-prompt results collected on the test set are shown below.

a)



b)

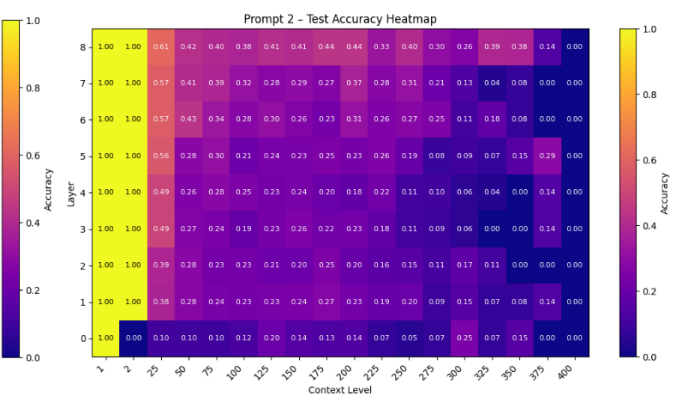
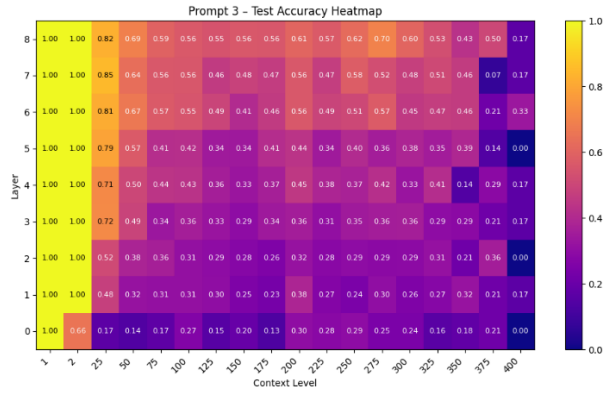
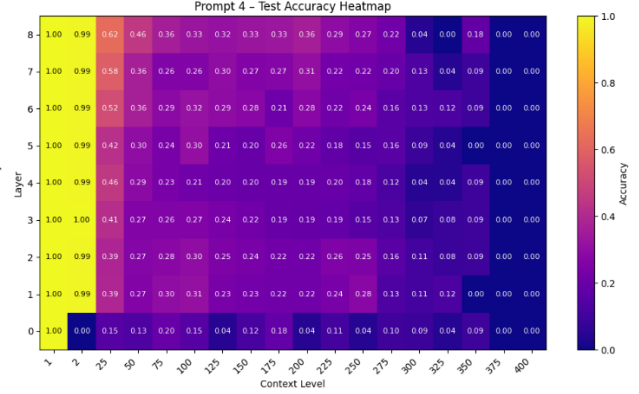


Figure 6.3: a) – j) Test accuracy heatmaps for prompts 1-10, respectively.

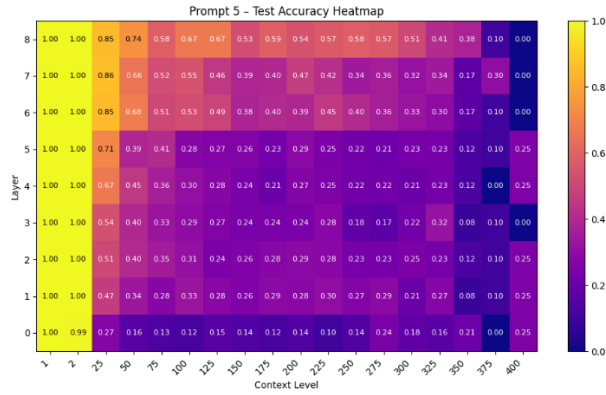
c)



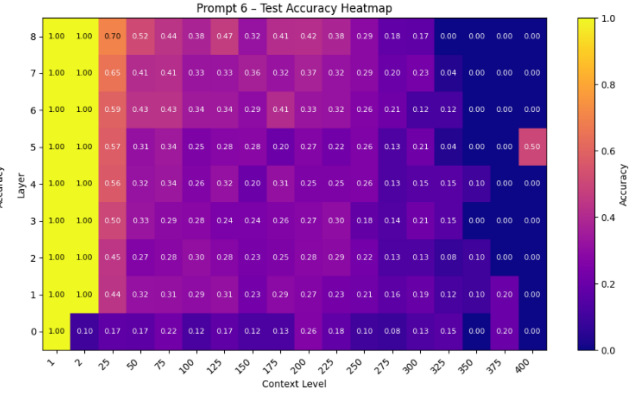
d)



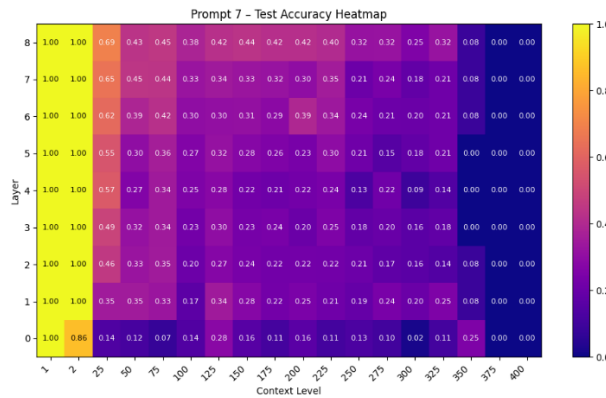
e)



f)



g)



h)

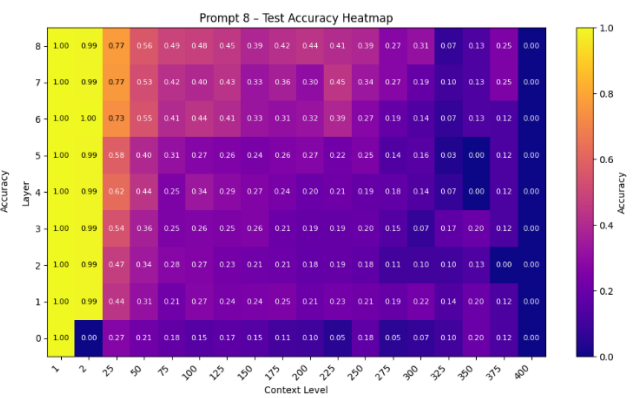
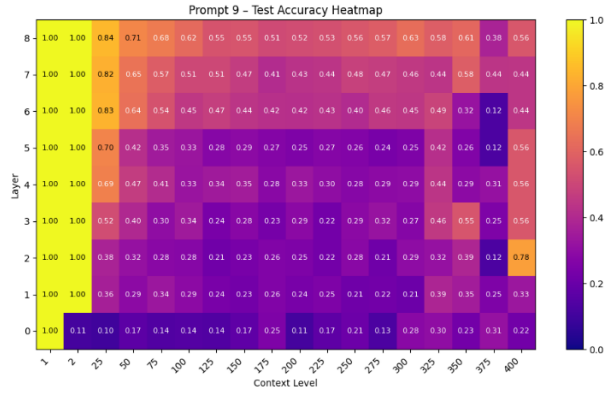


Figure 6.3, continued: a) – j) Test Accuracy Heatmaps for Prompts 1-10, respectively

i)



j)

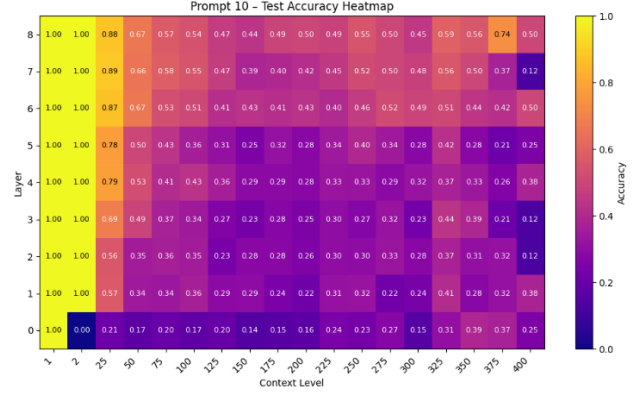
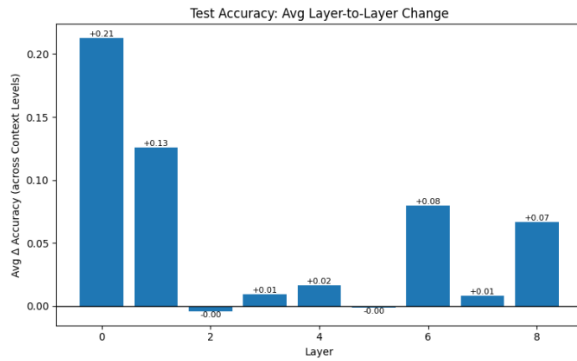


Figure 6.3, continued: a) – j) Test Accuracy Heatmaps for Prompts 1-10, respectively

III. Layer-to-Layer and Context Level-to-Context Level Patterns

To make layerwise and generation level patterns clearer, we performed an analysis of generation level-to-generation level and layer-to-layer changes with respect to the test accuracy. The results of these analyses are shown below.

a)



b)

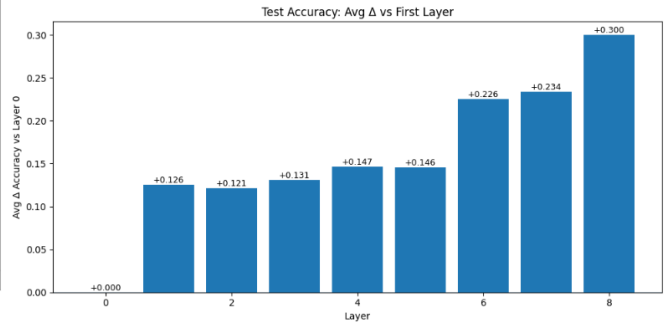
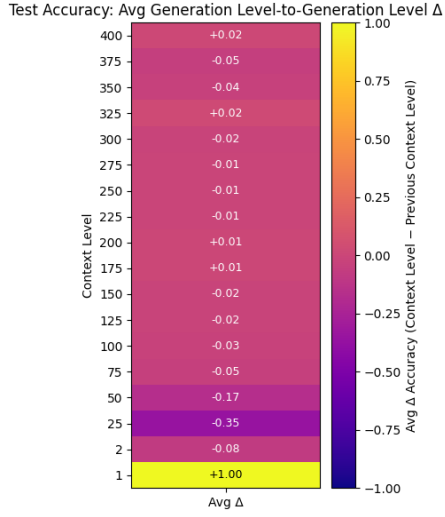


Figure 6.4: a) Average difference in test accuracy between consecutive layers. b) Average difference in test accuracy between current layer and layer 0.

a)



b)

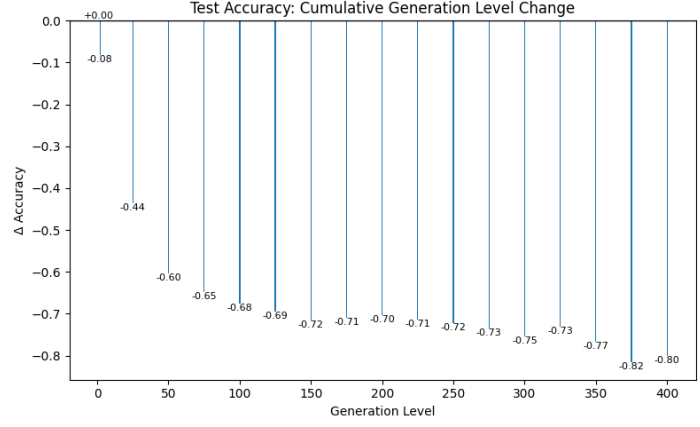


Figure 6.5: a) Average difference in test accuracy between consecutive generation levels. b) Average difference in test accuracy between current generation level and the first generation level.

IV. First-token Generations with Single Token Context

We now proceed to display the results of our second experiment, which focuses on analysis of first-token generation with minimal context.

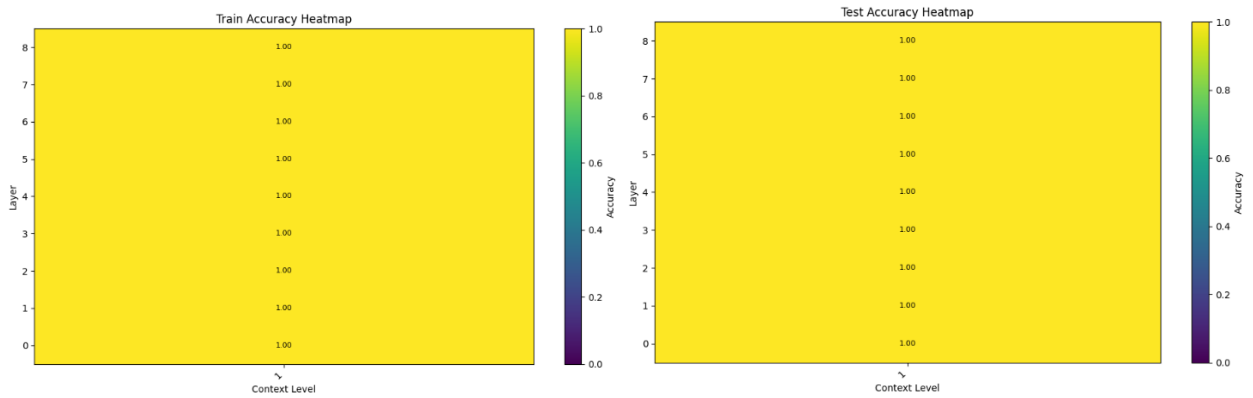


Figure 6.6: a) Heatmap of overall train and test accuracy across all layers when using XGBoost to perform prompt prediction on first token generation hidden states with single token context.

Here, we see perfect predictive accuracy across all layers. A brief overview of this phenomenon is presented in the “Discussion of Results” chapter of this paper.

V. Hyperparameter Optimization via Bayesian Search

The overall results concerning hyperparameter optimization on the train and test sets are shown below:

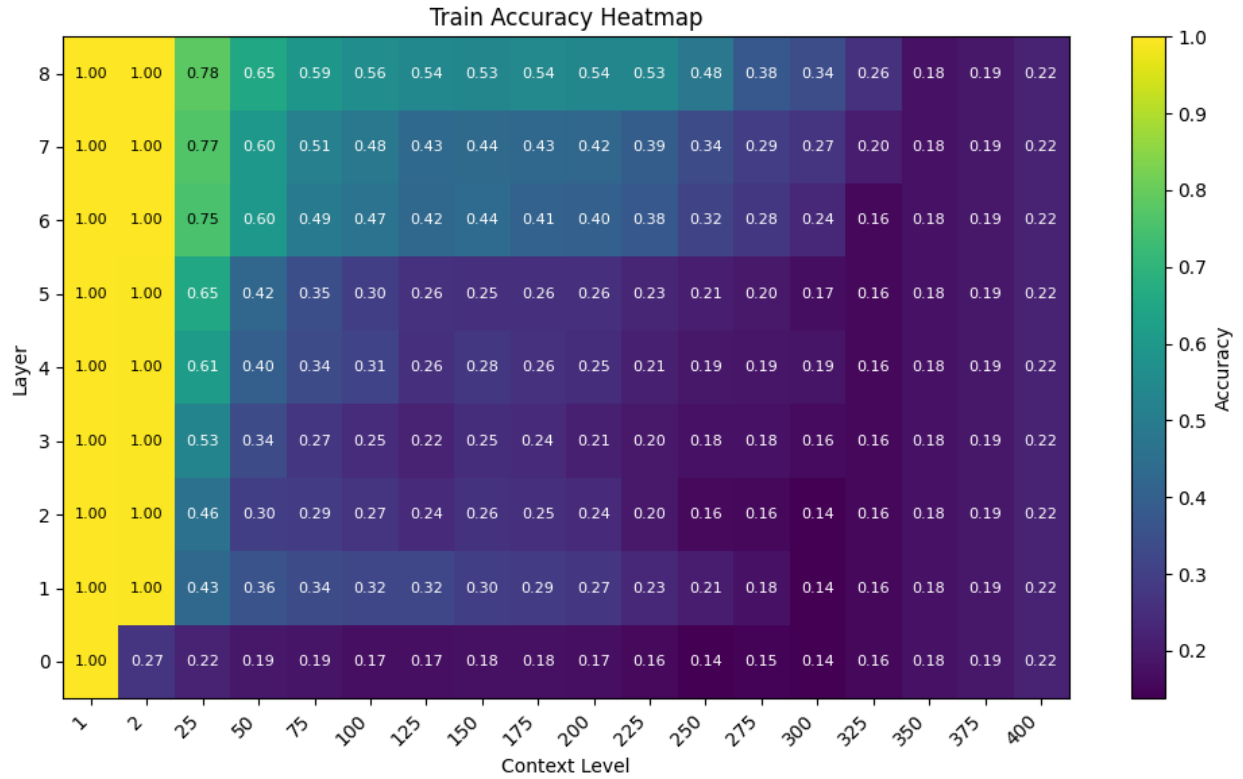


Figure 6.7: Heatmap of train accuracy after hyperparameter optimization is conducted.

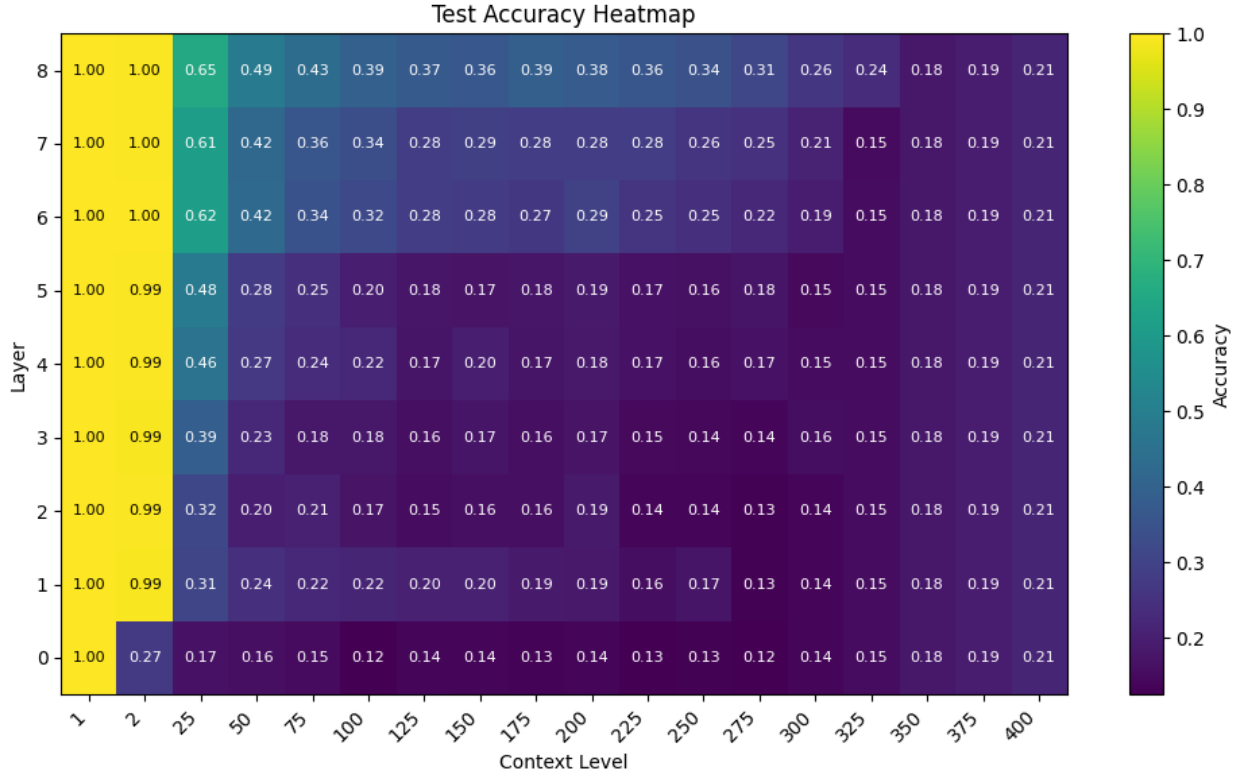


Figure 6.8: Heatmap of test accuracy after hyperparameter optimization is conducted.

VI. XGBoost Feature Importance (Hidden Dimension) Analysis

Finally, our investigation into hidden dimension importance is presented in the following section.

We begin by showing a heatmap of the most important hidden dimensions chosen by XGBoost when evaluating on the entire test set across all layers and generation levels. We do not include first token generations in the heatmap color encoding due to peculiarities in its feature dimensions, but the values are still provided here for reference.

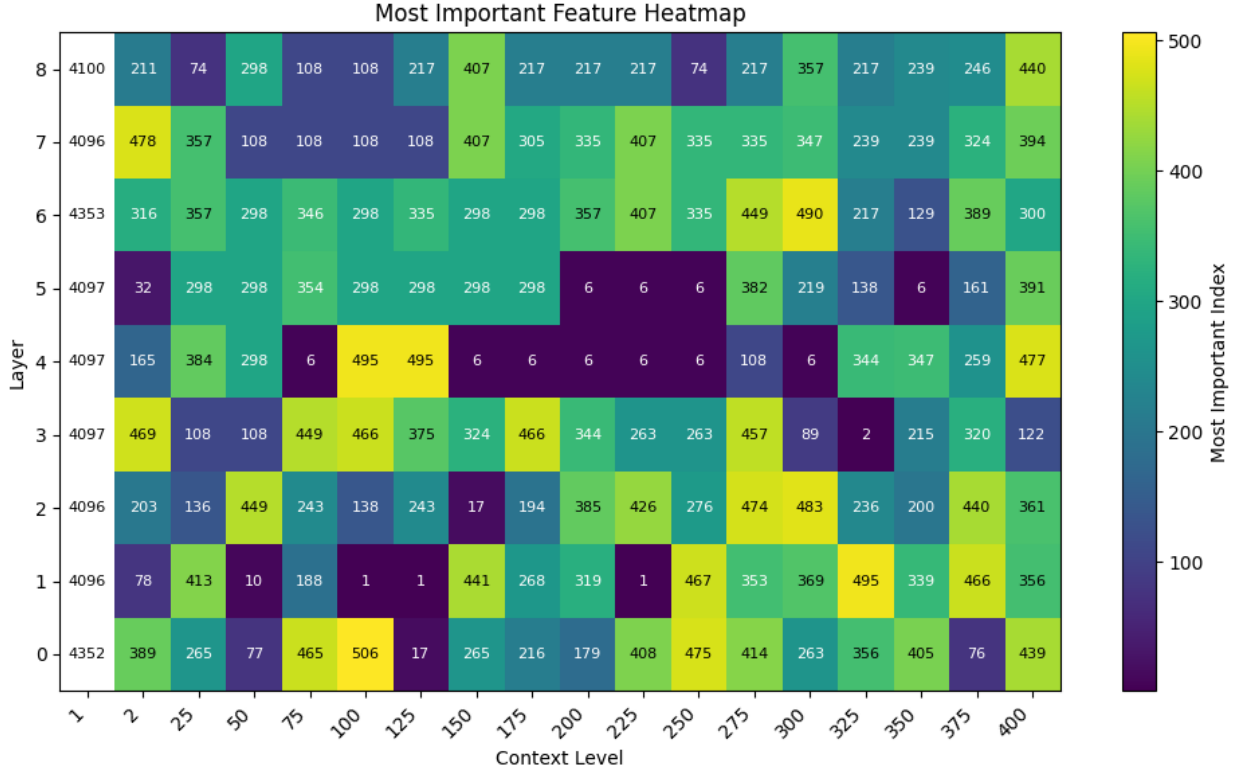


Figure 6.9: Heatmap of feature importances as determined by XGBoost when making predictions on the test set.

Because the hidden states used to collect results for first token generation were represented as a 4608-dimensional vector, we have that the hidden dimensions for the first generation level correspond to a certain token of context, and the actual hidden dimension relating to that token is the value shown divided by 9. We also observed during our analysis of Shapley values that there was always only one feature deemed important by XGBoost in determining the prompt from the first token generation hidden states, across all layers, as can be seen in the following bar plot. This differs from later token generations, in which, by comparison, there were multiple other hidden dimensions that were still deemed important.

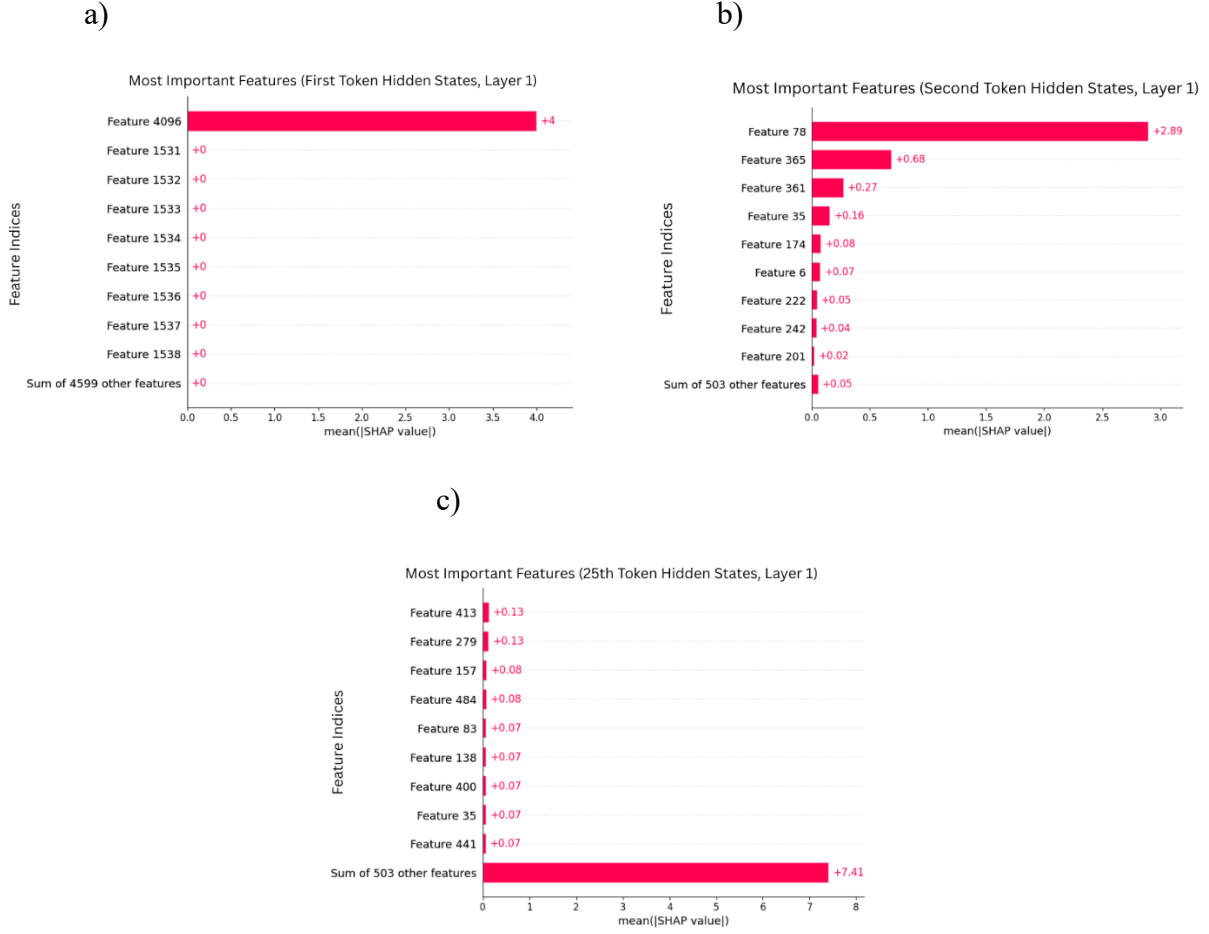


Figure 6.10: Comparison between important features determined using Shapley values. Shown here are the most important features determined when using hidden states collected at layer 1 during the generation of the a) first token vs. those collected during the generation of the b) 2nd and c) 25th tokens.

Next, we present heatmaps of the most important hidden dimensions chosen by XGBoost, as before, but now split by prompt.

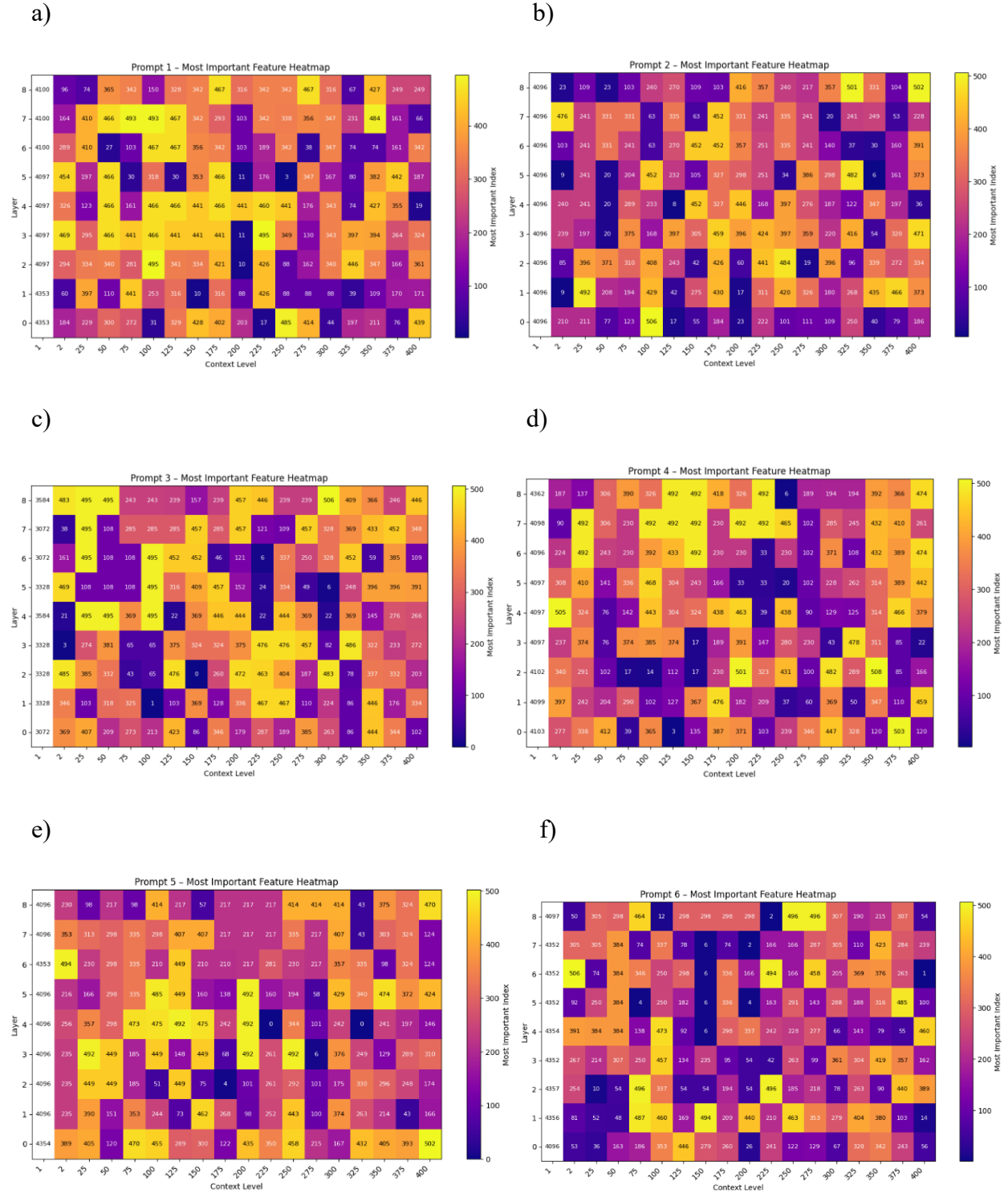


Figure 6.11: a) – j) Heatmap of most important features used for prediction on the test set across all layers and generation levels.

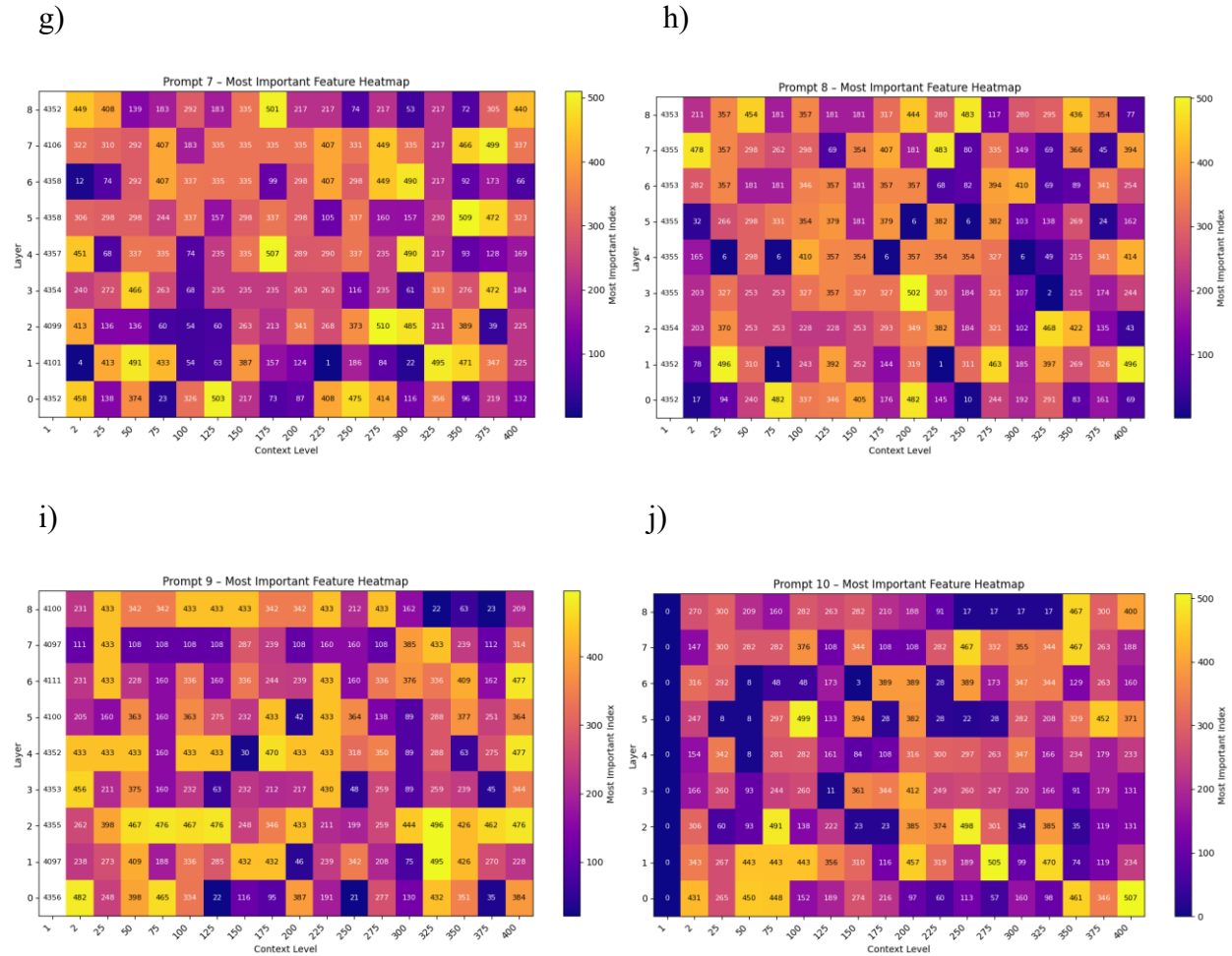


Figure 6.11, continued: a) – j) Heatmap of most important features used for prediction on the test set across all layers and generation levels.

Finally, we show a heatmap containing the mean of the five most important hidden dimensions chosen by XGBoost when evaluating on the entire test set for each layer and generation level. Here, we exclude the values for first token generation because there was always only one hidden dimension deemed important using those hidden states, as explained above.

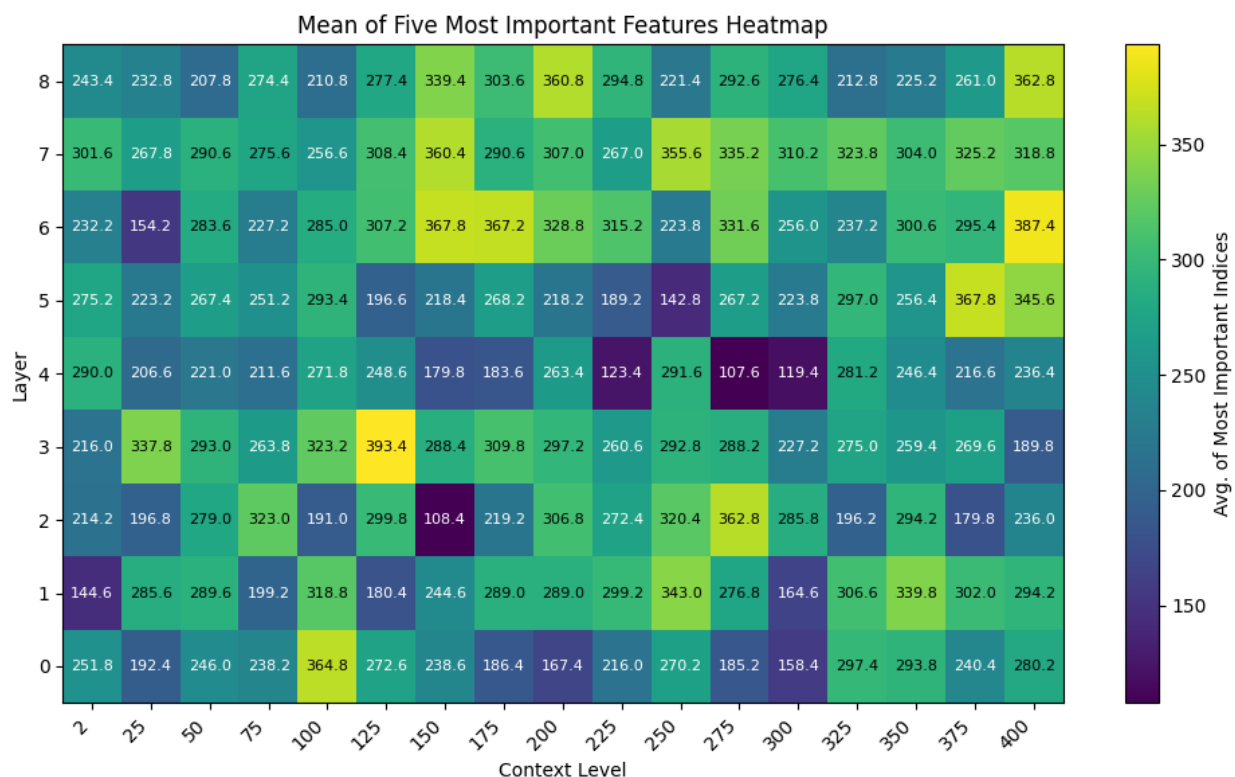


Figure 6.12: Heatmap showing the mean of the five most important features across generation levels 2-400 and all layers.

With this plot, we see a possible range of hidden dimensions that may be deemed “most important” when applying a binary mask on the chart based off low versus high index values. We see that most important hidden dimensions tend to fall in the mid range rather than at any index at either end of the spectrum.

Chapter 7 DISCUSSION OF RESULTS

The primary objective of this work was to investigate how information about an input prompt is preserved within the hidden states of an autoregressive transformer during text generation to understand better how a language model transforms input to output. By extracting hidden states from multiple layers and generation depths and applying a supervised classifier to recover the original prompt, we aimed to determine which layers contain the most useful information linking the model’s output to its input. Through a series of experiments using XGBoost, we analyzed predictive performance across layers, across generation levels, and across prompts, allowing us to examine how prompt information evolves during the generation process. From the plots in the section entitled “Experimental Results”, we derive the following conclusions about layer importance and generation length in storing and transforming information about the input.

One of the most striking observations from our experiments was the perfect predictive accuracy obtained when classifying prompts using the hidden states involved in generating the first token of the story. Across all layers, the classifier was able to correctly identify the prompt with one-hundred percent accuracy on both the training and test sets. This outcome, however, is explainable when considering how the first token is generated. During the initial forward pass, the model processes the entire prompt sequence simultaneously. As a result, we see that the hidden states contain direct representations of the prompt tokens themselves. Because these hidden states explicitly encode the prompt context, they contain sufficient information to uniquely identify which prompt produced the story. Consequently, the classifier can easily map hidden state patterns to their corresponding prompts.

However, this perfect predictability changes immediately after the first token generation. When the model generates the second token, the hidden states in the first layer (layer 0) exhibit a substantial drop in predictive performance. This decline can be explained by the nature of the autoregressive generation process. For tokens beyond the first, layer 0 receives only the most recently generated token as explicit input. In other words, the hidden states in this layer primarily represent an embedding of the previous token rather than the entire prompt. Because the prompt itself is no longer explicitly provided at this stage, the hidden states in layer 0 contain little direct information about the original input. As a result, prompt classification using these representations becomes much more difficult, leading to lower predictive accuracy.

Despite this drop in early layers, the experiments reveal a consistent and somewhat surprising trend across deeper layers. As representations move through the transformer stack, the predictive accuracy gradually increases. Later layers, particularly layers 6 through 8, consistently demonstrate significantly stronger prompt recoverability than earlier layers. This pattern appears across nearly all generation levels examined in the study. We see through a layer-to-layer analysis that prompt information is mostly gained in early layers and either lost or stabilized in the middle layers before being recovered and/or compounded in later layers.

The improvement in predictive accuracy in deeper layers suggests that prompt information is gradually reintroduced or reconstructed as hidden states pass through the network. One likely explanation for this phenomenon is the presence of the key-value (KV) cache used during autoregressive generation. While layer 0 processes only the current token, the attention

mechanism in later layers can access representations of earlier tokens stored in the cache, including those produced during the initial prompt processing step. These cached representations contain information about the prompt, which can be integrated into the current hidden state through the attention mechanism. As a result, later layers can incorporate contextual information that extends beyond the most recently generated token, effectively restoring some of the prompt-related information that was absent in earlier layers.

This pattern becomes particularly clear when examining intermediate generation steps such as the 25th or 50th token. At these stages, early layers again exhibit relatively poor prompt recoverability, while deeper layers show substantially higher predictive accuracy. The hidden states in these later layers appear to contain a combination of information from the current token and contextual information retrieved from the KV cache. Consequently, these layers provide the strongest signals for identifying the original prompt.

Another important finding concerns the effect of generation length on prompt recoverability. As the number of generated tokens increases, the predictive accuracy of the classifier generally decreases. We see the largest drops in predictive accuracy happen early, around the 25th token, followed by gradual degradation in later token generations. However, this degradation becomes especially noticeable in long-range generations exceeding approximately 250 to 300 tokens, resulting in little to no prompt information remaining in these tokens' hidden states. Even in deeper layers, where prompt information is typically strongest, classification accuracy gradually declines as generation continues.

In our analysis of feature importance, we also see a similar pattern emerging, whereby less importance is placed on individual hidden dimensions as more tokens are generated. This not only suggests that prompt context greatly diminishes but is also likely more spread out across hidden dimensions than in early generations.

These results together suggest that prompt information slowly diminishes as more tokens are generated and the context grows longer. Over time, the KV cache accumulates an increasing amount of information about newly generated tokens. While this growing context is necessary for maintaining narrative coherence, it may also dilute the relative influence of the original prompt within the hidden state representations. As later tokens become more strongly influenced by the evolving story context rather than the initial prompt, the hidden states contain fewer distinctive features that allow the classifier to recover the prompt accurately.

From a practical perspective, this observation has implications for how language models are used during inference. If prompt information gradually becomes less influential during long generations, it may be beneficial to periodically refresh the context by reintroducing the original prompt or restarting generation with updated context windows. For our language model, this refresh point occurs around 225–250 tokens, or around half the context window of our model (512), before prompt information degrades significantly. Such strategies could help ensure that generated text remains closely aligned with the intended prompt, especially in tasks requiring long outputs.

In addition to analyzing overall predictive accuracy, we also examined prompt-specific classification performance. While most prompts exhibited similar accuracy trends, some prompts were more easily identifiable than others, particularly in long-range generations. Preliminary analysis of data distribution in the stories we generated suggests that these differences in accuracy are not solely explained by dataset representation or sample imbalance for certain generation levels. For instance, prompt 8 displayed substantially degraded performance at longer generation depths despite being equally represented in the dataset compared to prompts 9 and 10, which maintained higher predictive accuracy. This indicates that other factors, such as narrative variability or token distribution, may influence how strongly prompt information persists in hidden state representations.

We posit that this variation may arise from differences in how well certain prompts are represented in the language model’s training data. For example, prompts describing common story characters such as “princess,” “dragon,” or “king” may have appeared more frequently in the dataset used to train the model, resulting in more distinctive internal representations. Alternatively, some prompts may produce stories with more consistent narrative structures, making their hidden state patterns easier for the classifier to distinguish.

To further understand the role of layer 0 in forming hidden state representations, we conducted an additional experiment focusing on the first token generation step. In our original experiment, hidden states for the first token included representations for every token in the prompt. To make this scenario more comparable to later generation steps, we extracted only the

hidden state corresponding to the final token of the prompt, producing a representation similar to the ones used during subsequent token generations.

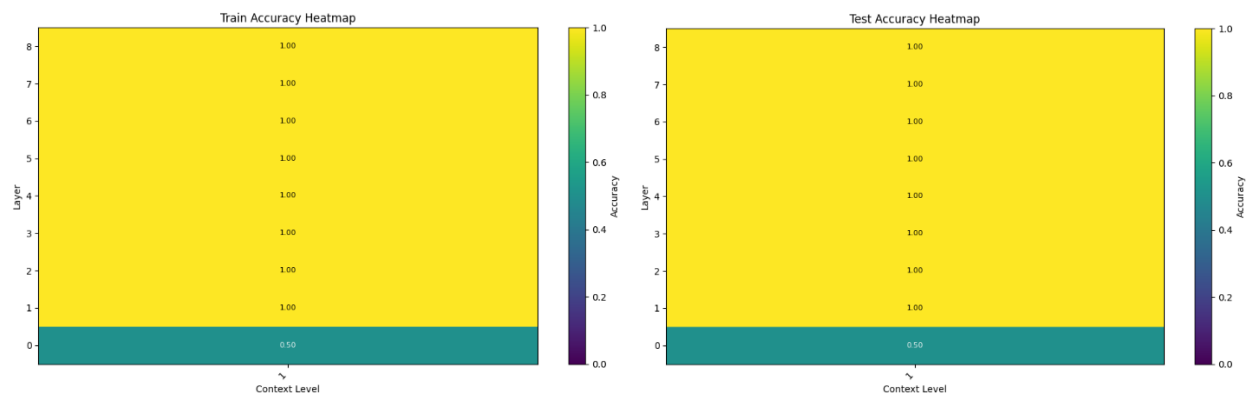
This modified experiment still produced perfect classification accuracy across all layers. This result can be explained by the fact that the final token of each prompt in our dataset is unique. Words such as “dragon,” “princess,” “children,” and “boy” have distinct embeddings that can easily be distinguished by the classifier. Consequently, the hidden states associated with these tokens remain highly predictive of the original prompt.

To test whether this effect was caused purely by the uniqueness of the final token, we conducted another experiment using two prompts that differed only slightly but ended with the same token:

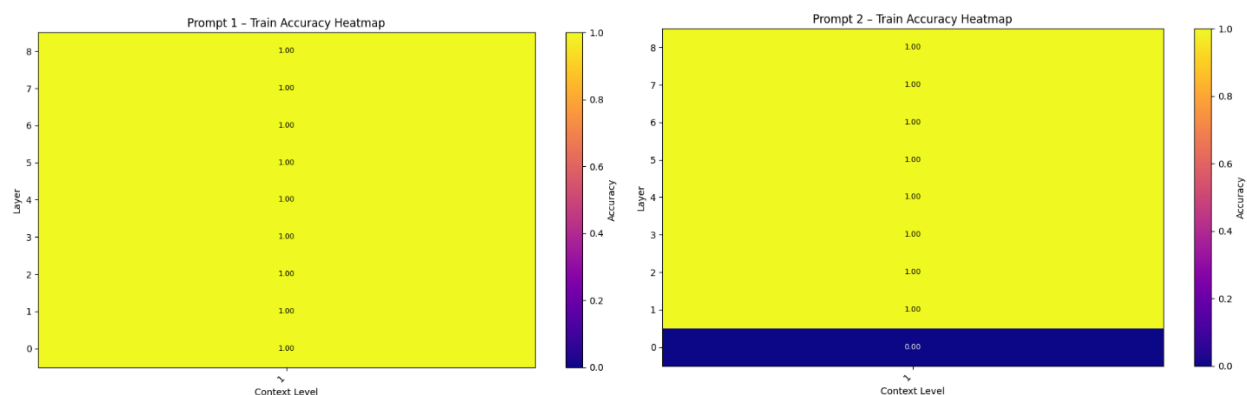
1. “Once upon a time there was a boy”
2. “Once upon a time there was a poor boy”

We then evaluate the predictive performance using the single token context hidden states collected during the first generation step based on these prompts. The results are shown below.

a)



b)



c)

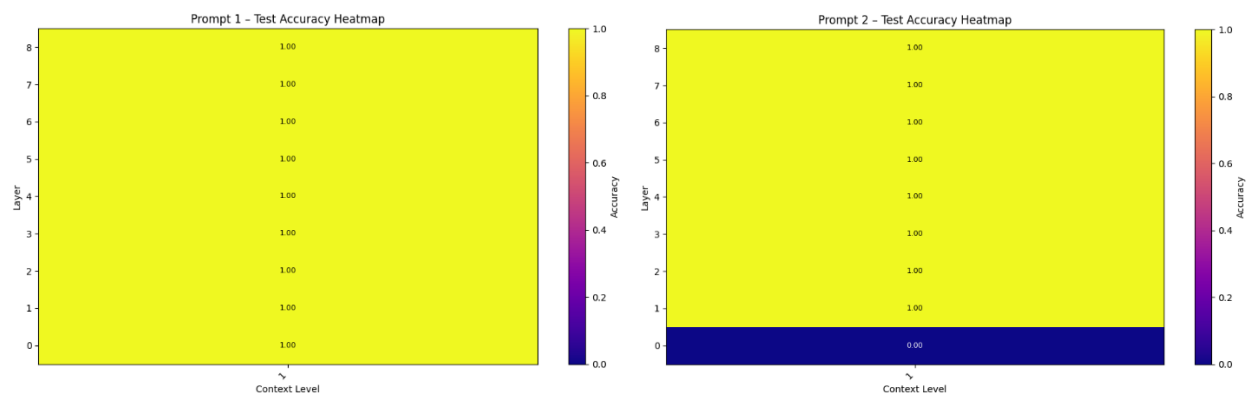


Figure 7.1: a) Heatmap of overall train and test accuracy across all layers for secondary experiment concerning first token generation with minimal context. b) Heatmap of prompt 1 train and test accuracy across all layers for secondary experiment concerning first token generation with minimal context. c) Heatmap of prompt 2 train and test accuracy across all layers for secondary experiment concerning first token generation with minimal context.

In this case, the final token of both prompts is identical (“boy”). When the classifier relied solely on the hidden states corresponding to this token, the results revealed a different pattern. In layer 0, where hidden states primarily represent direct token embeddings, the model could not distinguish between the two prompts. As a result, all examples were assigned to the same prompt class, producing an overall classification accuracy of fifty percent. However, later layers demonstrated significant ability in recovering prompt information. Beginning in layer 1 and continuing through later layers, classification accuracy returned to one hundred percent. This indicates that additional contextual information beyond the final token embedding is incorporated into hidden states as they pass through the transformer layers. These deeper layers appear to integrate information about earlier tokens in the prompt, such as the word “poor” in the second prompt, allowing the classifier to distinguish between otherwise similar prompts.

Analyzing the initial plots showing overall train and test accuracy, we see that performance degrades significantly between our train and test sets, which suggests overfitting to the training set. To ensure that the patterns observed through classification on the test set also applied to the training set, we conducted additional experiments involving hyperparameter optimization. Using the Optuna framework, we performed a Bayesian search to identify XGBoost hyperparameters that reduce model complexity while maintaining strong predictive performance. Early stopping and regularization techniques were also applied to prevent the classifier from memorizing the training data (Chen & Guestrin, 2016). After these adjustments, the overall trends observed in the training set closely matched those in the test set, confirming that the results reflect genuine patterns in the hidden state representations.

Finally, we investigated which hidden state dimensions were most important for classification using SHAP (SHapley Additive exPlanations) analysis. While the specific dimensions identified as important varied across experiments, a noteworthy pattern occasionally emerged in which the same hidden dimensions remained highly important across multiple layers and generation levels. In some cases, the most influential dimension was either consistent or alternated among a small set of dimensions across consecutive layers. This suggests that certain hidden units may play a persistent role in encoding information that connects the prompt to the generated output.

In addition to this, we see that particular prompts have greater importance given to early dimensions, especially in the generation of the first token, because of uniquenesses in the prompt structure itself. For example, we see in the prompt-specific feature importance heatmaps that hidden dimension 0, corresponding to the second token of prompt context, is prominent in determining prompt 10 from the first generation step hidden states. This is because the hidden states for prompt 10 are indexed starting only from the second token of context to match the dimensions of other prompts' hidden states. This causes a difference in the initial content of the hidden states for prompt 10 that allows it to be differentiated from the hidden states generated using other prompts. Another occurrence of this is noticeable in prompt 3, whereby a hidden dimension corresponding to the second to last token of the prompt is deemed the most important. This is because prompt 3, as opposed to other prompts, has an inherently different lexical structure, containing a numerical modifier that allows it to be easily differentiated via just a subset of its hidden states.

Although individual hidden state dimensions are difficult to interpret directly, the presence of these recurring patterns indicates that specific components of the representation space may be responsible for maintaining prompt-related information throughout the generation process. Further investigation into these dimensions could reveal how prompt features are encoded and transformed across layers.

Our experiments reveal several key insights into how autoregressive transformers maintain information about their input prompts during text generation. Prompt information is strongly encoded in the hidden states used to generate the first token, but this information becomes less explicit in early layers during subsequent token generations. Deeper layers recover and integrate prompt-related information through attention mechanisms and cached representations, making them the most informative layers for prompt identification. However, as generation length increases, prompt information gradually diminishes as new context accumulates. These findings contribute to a deeper understanding of how transformer architectures propagate input information through their hidden representations and highlight the set of layers most responsible for linking prompts to generated outputs.

Chapter 8 RELATED WORK

Understanding how language models internally represent and transform information has become an important research direction as large neural architectures have grown increasingly complex. While modern transformer-based language models demonstrate remarkable performance across a wide range of tasks, their internal mechanisms remain difficult to interpret. In this section, we review prior work related to transformer architectures, interpretability in neural language models, probing methods for analyzing internal representations, and approaches to model compression and layer analysis.

Modern language models are primarily built on the transformer architecture introduced by Vaswani et al. (2017). Transformers rely on self-attention mechanisms to model dependencies between tokens in a sequence without relying on recurrent or convolutional operations. By stacking multiple layers of attention and feedforward networks, transformers can build hierarchical representations of text that capture both syntactic and semantic relationships.

Subsequent work extended the transformer architecture to large-scale language modeling tasks. Radford et al. (2019) demonstrated that autoregressive transformers trained on large corpora can perform a variety of natural language tasks without task-specific training. More recent models, such as the LLaMA family of models (Touvron et al., 2023), further improved efficiency and scalability, enabling high-performing language models to be trained with fewer computational resources.

While the transformer architecture is well understood mathematically, the representations learned within its hidden layers remain largely opaque. As a result, many recent studies have focused on analyzing how information is encoded within these internal representations and how it changes across layers.

The interpretability of neural language models has become a significant research topic as these systems are increasingly deployed in real-world applications. Interpretability research aims to identify the mechanisms through which deep neural networks process information and produce predictions.

One influential line of work explores the concept of mechanistic interpretability, which seeks to understand neural networks by analyzing the computational circuits implemented by their components. Elhage et al. (2021) introduced a mathematical framework for studying transformer circuits, showing how attention heads and feedforward layers can be analyzed as structured computational modules. Subsequent work by Wang et al. (2022) demonstrated that specific transformer circuits can be identified that perform well-defined linguistic tasks, such as indirect object identification.

These studies suggest that meaningful patterns of computation may emerge within transformer layers, even though the models themselves are trained through gradient-based optimization rather than explicit algorithm design. However, identifying such circuits remains challenging due to the high dimensionality of hidden representations and the large number of interacting components within transformer networks.

Another important line of research focuses on representation superposition, which describes how neural networks may encode multiple features within overlapping subspaces of the hidden representation. Elhage et al. (2022) showed that neural networks can represent many features using fewer dimensions by combining them in superposition. This phenomenon complicates interpretability because individual hidden dimensions may not correspond directly to single interpretable concepts.

A widely used approach for analyzing neural representations is the use of probing classifiers. In this framework, a simple supervised model is trained to predict linguistic or contextual information from hidden states produced by a language model. If the probe can accurately recover a specific property, it suggests that the property is encoded within the representation (Belinkov, 2022).

Conneau et al. (2018) introduced probing tasks for evaluating sentence embeddings, demonstrating that neural representations can capture a variety of linguistic features including syntactic structure and semantic information. Similarly, Hewitt and Manning (2019) proposed structural probes that reveal how syntactic trees can be recovered from hidden representations in language models.

Layer-wise probing has also been used to analyze how linguistic information emerges across transformer layers. Tenney et al. (2019) found that different layers of the BERT model appear to specialize in different linguistic tasks, with early layers capturing surface-level features

and later layers encoding higher-level semantic information. Clark et al. (2019) further examined the attention mechanisms in BERT and showed that certain attention heads learn interpretable patterns related to syntax and coreference.

These probing methods provide a useful framework for studying the informational content of hidden states. However, they do not necessarily reveal how this information is used internally by the model during generation.

Another research direction focuses on analyzing how information propagates through transformer layers. Abnar and Zuidema (2020) introduced methods for quantifying attention flow in transformers, allowing researchers to estimate how strongly tokens influence one another through the attention mechanism.

Geva et al. (2021) further showed that feedforward layers in transformers can act as key-value memories, storing associations between input patterns and output activations. This perspective suggests that transformer layers may function as distributed memory systems that retrieve and transform contextual information during inference.

In addition to analyzing attention patterns, some studies have examined the possibility of reconstructing input information from hidden states. Carlini et al. (2021) demonstrated that language models may inadvertently memorize and reveal training data, indicating that substantial information about inputs can remain encoded within internal representations.

Our work builds upon these insights by directly evaluating how strongly hidden states encode information about the original input prompt during autoregressive generation. By applying a classifier to hidden state vectors across layers and generation depths, we aim to quantify the extent to which prompt information is preserved throughout the generation process.

Beyond interpretability, another relevant line of research investigates the relative importance of different transformer components. Many studies have shown that large neural networks contain significant redundancy and can be compressed without major losses in performance.

Michel et al. (2019) demonstrated that many attention heads in transformers can be removed with little impact on model accuracy, suggesting that only a subset of heads perform critical computations. Similarly, Fan et al. (2020) introduced LayerDrop, a training technique that randomly drops transformer layers during training to produce models that are robust to layer removal.

Model compression techniques such as knowledge distillation (Hinton et al., 2015) and parameter-efficient fine-tuning methods like LoRA (Hu et al., 2021) further highlight the potential for reducing model complexity while maintaining performance. These approaches suggest that not all components of large language models contribute equally to their predictive capabilities.

Understanding which layers contain the most informative representations is therefore valuable not only for interpretability but also for model optimization. Our study contributes to this area by identifying which layers contain the strongest signals linking generated tokens to their originating prompts.

Prior research has demonstrated that transformer language models encode a rich set of linguistic and contextual features within their hidden states. However, relatively little work has examined how prompt information is preserved during autoregressive generation or how this information evolves across generation depth. By combining hidden state extraction with supervised classification, our work provides a quantitative framework for analyzing the relationship between input prompts and generated outputs across transformer layers.

Chapter 9 FUTURE WORK

While our study provides insight into how prompt information is represented within transformer hidden states during autoregressive generation, several limitations remain. These limitations arise from the scale of the model studied, the structure of the dataset used for generation, and the methodological assumptions inherent in probing-based analysis. Addressing these limitations offers several promising directions for future work.

One primary limitation of this work is the relatively small scale of the model analyzed. Due to resource constraints, our experiments were conducted using the LlamaTales-33M model, a lightweight transformer designed to replicate the architectural structure of larger language models while remaining computationally tractable for interpretability experiments. While this design choice enables efficient extraction and analysis of hidden states across thousands of generated stories, it also raises questions regarding the generalizability of our findings to larger language models.

Large-scale transformer models often exhibit emergent behaviors that are not present in smaller models. Previous research has demonstrated that representational properties, reasoning capabilities, and internal circuits can change substantially as model size increases. Consequently, it remains unclear whether the same layer-wise patterns observed in this study, such as the increasing recoverability of prompt information in deeper layers or the gradual degradation of prompt signals during long-range generation, would appear in models with billions of parameters.

Future work should therefore replicate this analysis on larger transformer models, including models in the LLaMA, GPT, or Mistral families of LLMs. Conducting similar hidden state analyses on models with deeper architectures and larger embedding dimensions may reveal whether prompt information propagates differently in networks with greater representational capacity. Additionally, larger models may rely more heavily on specialized attention heads or circuits that maintain long-range context, which could alter the patterns observed in our experiments.

Another limitation arises from the structure of the dataset used in our experiments. The prompts used to generate stories were intentionally simple and followed a consistent template (“Once upon a time there was a ...”). This design ensured that the generated outputs remained within the vocabulary and narrative style learned by the model, allowing us to generate a large dataset of coherent stories suitable for analysis. However, this uniformity may also simplify the prompt classification task.

In particular, the final token of each prompt in our dataset is unique (e.g., “dragon”, “princess”, “king”, “wolf”), making it relatively easy for the classifier to identify the prompt based solely on token embeddings. This characteristic partially explains the perfect predictive performance observed when classifying hidden states associated with the generation of the first token. Although we attempted to address this issue by introducing prompts that share the same final token (e.g., “boy” vs. “poor boy”), the dataset remains relatively constrained in its diversity.

Future studies could expand this analysis by using prompts with greater lexical overlap and more varied syntactic structures. For example, prompts that differ primarily in semantic meaning rather than surface tokens may provide a more challenging classification task and could reveal more subtle representational differences across layers. Similarly, analyzing prompts that include longer contextual descriptions or narrative setups may provide insight into how transformers encode and retain complex prompt information during extended generation.

A third limitation presents itself in our study’s narrow focus, primarily on hidden state representations without directly analyzing the internal attention patterns that contribute to these representations. However, attention mechanisms and the key–value (KV) cache play a central role in autoregressive generation by allowing later tokens to access information from earlier tokens in the sequence.

Although our results suggest that deeper layers regain prompt information by accessing cached representations from earlier tokens, we did not explicitly measure how attention distributions evolve across layers and generation depths. Understanding how attention weights shift over time could provide deeper insight into why prompt information becomes less recoverable during long-range generation.

Future research could combine hidden state analysis with attention flow analysis to examine how information about the prompt propagates through the KV cache. For example, tracking which tokens receive the highest attention weights during generation may reveal whether the model gradually shifts its focus from the original prompt to recently generated

tokens. Such analyses could help explain the observed decline in prompt recoverability at later generation stages.

Another potential research direction concerns our SHAP-based feature analysis, which identified individual hidden dimensions that occasionally appeared to play a significant role in prompt classification. However, interpreting the meaning of these dimensions remains difficult. Neural network representations are highly distributed, and individual hidden units rarely correspond to single interpretable concepts.

Furthermore, the importance of specific dimensions varied across layers and generation levels, suggesting that prompt information may be encoded in a dynamic and distributed manner rather than in fixed units. While SHAP analysis provides useful insight into which features contribute most strongly to classification performance, it does not directly reveal the semantic or functional meaning of those features.

Future work could explore methods for identifying interpretable feature directions within hidden representations. Techniques such as principal component analysis, sparse autoencoders, or representation steering methods may help uncover higher-level structures within the hidden state space. Understanding how prompt information is encoded within these structures could lead to more interpretable models and improved debugging tools for large language models.

Finally, our results suggest that prompt information gradually degrades during long-range generation, particularly just beyond half the size of the model’s context window. This

observation has implications for how language models manage long contexts during inference. However, our study does not directly test mechanisms that might mitigate this degradation.

Future work could explore strategies for preserving prompt influence during long generations. For example, techniques such as prompt reinforcement, sliding context windows, or periodic prompt re-injection could be evaluated to determine whether they improve alignment between generated text and the original prompt. Additionally, comparing models with different context window sizes may reveal whether longer context capacities help preserve prompt information over extended generations.

Despite these limitations, our work provides a useful empirical framework for studying how input information is represented within transformer hidden states during autoregressive generation. By combining hidden state extraction with supervised classification and feature attribution analysis, we demonstrate that prompt information is strongly encoded in early generation stages, partially reconstructed in deeper layers, and gradually diluted as generation continues.

Future research that extends this framework to larger models, more diverse prompts, and applying alternative methods may provide deeper insight into the internal mechanisms that connect input prompts to generated outputs in transformer-based language models.

REFERENCES

- Abnar, S., & Zuidema, W. (2020). Quantifying attention flow in transformers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (pp. 4190–4197). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.385>
- Blalock, D., Ortiz, J. J. G., Frankle, J., & Gutttag, J. (2020). What is the state of neural network pruning? Proceedings of Machine Learning and Systems, 2, 129–146. <https://doi.org/10.48550/arXiv.2003.03033>
- Belinkov, Y. (2022). Probing classifiers: Promises, shortcomings, and advances. Computational Linguistics, 48(1), 207–219. https://doi.org/10.1162/coli_a_00422
- Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., Oprea, A., & Raffel, C. (2021). Extracting training data from large language models. In Proceedings of the 30th USENIX Security Symposium (pp. 2633–2650). <https://doi.org/10.48550/arXiv.2012.07805>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Clark, K., Khandelwal, U., Levy, O., & Manning, C. D. (2019). What does BERT look at? An analysis of BERT’s attention. In Proceedings of the 2019 ACL Workshop on BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP (pp. 276–286). Association for Computational Linguistics. <https://doi.org/10.18653/v1/W19-4828>
- Conneau, A., Kruszewski, G., Lample, G., Barrault, L., & Baroni, M. (2018). What you can cram into a single vector: Probing sentence embeddings for linguistic properties. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (pp. 2126–2136). <https://doi.org/10.48550/arXiv.1805.01070>
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608. <https://arxiv.org/abs/1702.08608>
- Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D.,

- Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., & Olah, C. (2021). A mathematical framework for transformer circuits. Transformer Circuits Thread. <https://transformer-circuits.pub/2021/framework/index.html>
- Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., & Olah, C. (2022). Toy models of superposition (arXiv:2209.10652) [Preprint]. arXiv. <https://arxiv.org/abs/2209.10652>
- Fan, A., Grave, E., & Joulin, A. (2020). Reducing transformer depth on demand with structured dropout. In International Conference on Learning Representations (ICLR). <https://doi.org/10.48550/arXiv.1909.11556>
- Fantozzi, P., & Naldi, M. (2024). The Explainability of Transformers: Current Status and Directions. *Computers*, 13(4), 92. <https://doi.org/10.3390/computers13040092>
- Frankle, J., & Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks. International Conference on Learning Representations (ICLR). <https://doi.org/10.48550/arXiv.1803.03635>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer feed-forward layers are key-value memories. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (pp. 5484–5495). <https://doi.org/10.48550/arXiv.2012.14913>
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *Advances in Neural Information Processing Systems*, 35, 507–520. <https://doi.org/10.48550/arXiv.2207.08815>
- Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. International Conference on Learning Representations (ICLR). <https://doi.org/10.48550/arXiv.1510.00149>
- Hewitt, J., & Manning, C. D. (2019). A structural probe for finding syntax in word representations. In Proceedings of the 2019 Conference of the North American Chapter of the

Association for Computational Linguistics (NAACL) (pp. 4129–4138).
<https://doi.org/10.18653/v1/N19-1419>

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531. <https://arxiv.org/abs/1503.02531>

Hsieh, W., Bi, Z., Jiang, C., Liu, J., Peng, B., Zhang, S., Pan, X., Xu, J., Wang, J., Chen, K., Feng, P., Wen, Y., Song, X., Wang, T., Liu, M., Yang, J., Li, M., Jing, B., Ren, J., Song, J., Tseng, H.-M., & Zhang, Y. (2024). A comprehensive guide to explainable AI: From classical models to LLMs. arXiv. <https://doi.org/10.48550/arXiv.2412.00800>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. International Conference on Learning Representations (ICLR). <https://doi.org/10.48550/arXiv.2106.09685>

Kim, J., El Halabi, M., Ji, M., & Song, H. O. (2024). LayerMerge: Neural network depth compression through layer pruning and merging. arXiv. <https://arxiv.org/abs/2406.12837>

Lee, I., & Berg-Kirkpatrick, T. (2025). Readability $\neq$ learnability: Rethinking the role of simplicity in training small language models. arXiv. <https://doi.org/10.48550/arXiv.2510.13915>

Linardatos, P., Papastefanopoulos, V., & Kotsiantis, S. (2021). Explainable AI: A review of machine learning interpretability methods. *Entropy*, 23(1), 18.
<https://doi.org/10.3390/e23010018>

Lipton, Z. C. (2016). The mythos of model interpretability. *Communications of the ACM*, 61(10), 36–43. <https://doi.org/10.1145/3233231>

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
https://papers.nips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

Lundberg, S. M., Erion, G., & Lee, S.-I. (2018). Consistent individualized feature attribution for tree ensembles. arXiv preprint arXiv:1802.03888. <https://arxiv.org/abs/1802.03888>

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67. <https://doi.org/10.1038/s42256-019-0138-9>

Michel, P., Levy, O., & Neubig, G. (2019). Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems* (Vol. 32). <https://doi.org/10.48550/arXiv.1905.10650>

Mumuni, A., & Mumuni, F. (2025). Explainable artificial intelligence (XAI): From inherent explainability to large language models. *arXiv*. <https://doi.org/10.48550/arXiv.2501.09967>

Palikhe, A., Wang, Z., Yin, Z., Guo, R., Duan, Q., Yang, J., & Zhang, W. (2025). Towards transparent AI: A survey on explainable language models. *arXiv*. <https://doi.org/10.48550/arXiv.2509.21631>

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Technical Report*. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Sun, W., Song, X., Li, P., Yin, L., Zheng, Y., & Liu, S. (2025). The curse of depth in large language models. *arXiv*. <https://doi.org/10.48550/arXiv.2502.05795>

Tenney, I., Das, D., & Pavlick, E. (2019). BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 4593–4601). <https://doi.org/10.48550/arXiv.1905.05950>

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. <https://arxiv.org/abs/2302.13971>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>

Wang, K., Variengien, A., Conmy, A., Shlegeris, B., & Steinhardt, J. (2022). Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. arXiv preprint arXiv:2211.00593. <https://arxiv.org/abs/2211.00593>

Zhang, Y., Dong, Y., & Kawaguchi, K. (2024). Investigating layer importance in large language models. arXiv. <https://doi.org/10.48550/arXiv.2409.14381>

Zhao, H., Chen, H., Yang, F., Liu, N., Deng, H., Cai, H., Wang, S., Yin, D., & Du, M. (2023). Explainability for large language models: A survey. arXiv. <https://doi.org/10.48550/arXiv.2309.01029>